

IT3501- Full Stack Web Development

Unit - I

Understanding the Basic Web Development Framework:

Web:

- ❖ The full form of WWW is the World Wide Web. WWW is also called a Web and it is a catalogue of an order of all websites connected to the worldwide Internet.
- ❖ It is an information system in which linked hypertext data and resources are accessed over the Internet.
- ❖ In simply it refers to Websites and Web Pages or anything that works over the internet.

Development: Refers to building the application from scratch.

Web Development:

- ❖ Web development refers to the creating, building, and maintaining of websites. It includes aspects such as web design, web publishing, web programming, and database management.
- ❖ It is the creation of an application that works over the internet i.e. websites.

Web Development can be classified into two ways:

Frontend Development

- ❖ The part of a website where the user interacts directly is termed as front end. It is also referred to as the 'client side' of the application.

Popular Frontend Technologies

- ❖ **HTML:** HTML stands for HyperText Markup Language. It is used to design the front end portion of web pages using markup language. It acts as a skeleton for a website since it is used to make the structure of a website.
- ❖ **CSS:** Cascading Style Sheets fondly referred to as CSS is a simply designed language intended to simplify the process of making web pages presentable. It is used to style our website.
- ❖ **JavaScript:** JavaScript is a scripting language used to provide a dynamic behavior to our website.
- ❖ **Bootstrap:** Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular CSS

- ❖ framework for developing responsive, mobile-first websites. Nowadays, the websites are perfect for all browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones).
 - Bootstrap 4
 - Bootstrap 5

Backend Development

Backend is the server side of a website. It is part of the website that users cannot see and interact with. It is the portion of software that does not come in direct contact with the users. It is used to store and arrange data.

Popular Backend Technologies

- **PHP:** PHP is a server-side scripting language designed specifically for web development.
- **Java:** Java is one of the most popular and widely used programming languages. It is highly scalable.
- **Python:** Python is a programming language that lets you work quickly and integrate systems more efficiently.
- **Node.js:** Node.js is an open source and cross-platform runtime environment for executing JavaScript code outside a browser.

Full-stack Development

- ❖ Back-end website development and front-end development are carried out by different professionals with expertise in each.
- ❖ When the web solution is developed by a single developer who has experience with both front-end and back-end, it is called full-stack development.

Web Development Framework:

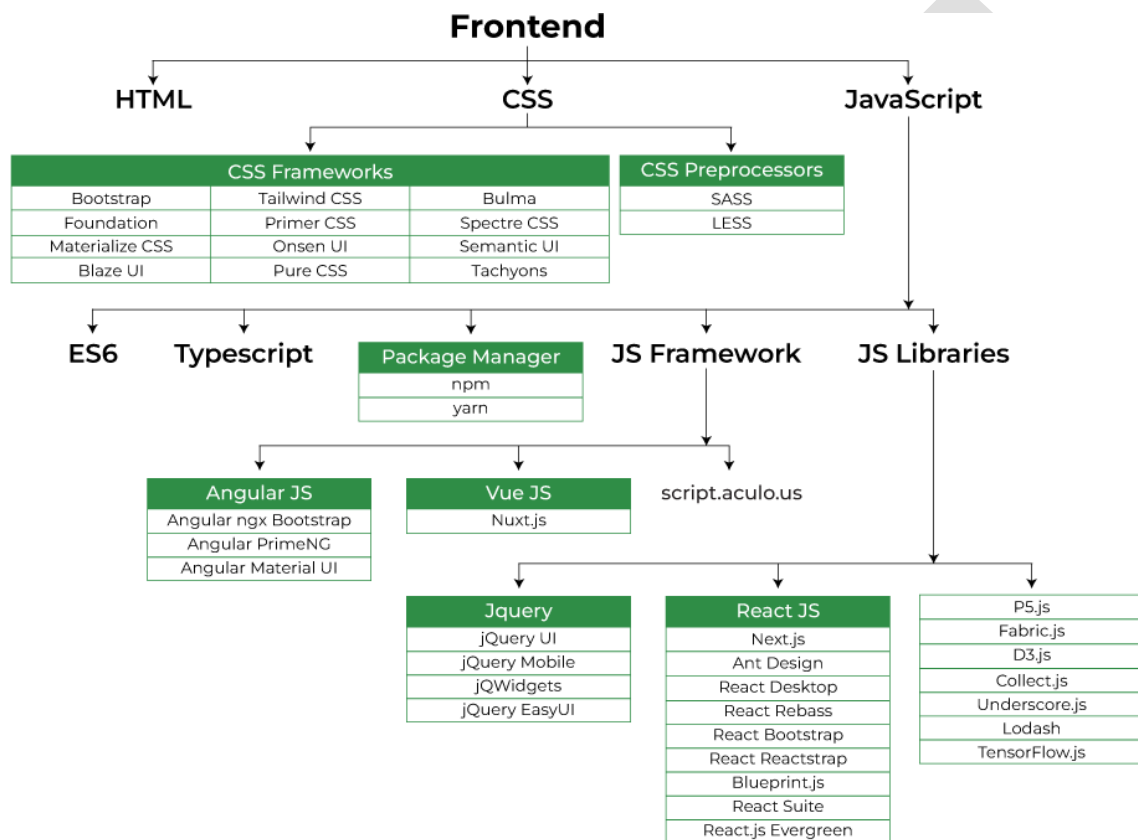
- ❖ A web development framework is a set of resources and tools for software developers to build and manage [web applications](#), [web services](#) and websites, as well as to develop application programming interfaces ([APIs](#)).
- ❖ Web development frameworks are also referred to as web application frameworks or simply web frameworks.
- ❖ Web development frameworks enable developers to build applications that can run on well-known technology [stacks](#) such as the [Linux](#), [Apache](#), [MySQL](#) and [PHP](#) ([LAMP](#)) stack

- ❖ Most frameworks provide a wide range of features and functionality that help streamline application development.
- ❖ Because web development frameworks are so comprehensive in scope, they offer development teams several important benefits, including the following:
- ❖ Developers can build applications faster and more efficiently because they write less code, reuse code, and contend with fewer errors and [bugs](#).
- ❖ Many frameworks are [open source](#) and backed by strong developer communities that help to optimize the code and reduce errors, leading to better performance and reliability, as well as easier maintenance.
- ❖ Many web development frameworks are widely used, adhere to industry standards and are backed by strong developer communities.
- ❖ The continuous vetting and improvements that this provides results in better security.
- ❖ Developers also avoid many of the risks that come with building applications from scratch.
- ❖ Web development frameworks speed up application development, reduce errors, simplify debugging and increase reliability.
- ❖ Many of them are also open source and free. Taken together, the factors can significantly reduce overall development costs.
- ❖ A web development framework also provides the foundation and system-level services necessary to support a content management system ([CMS](#)).
- ❖ A content management system is an application built on top of the development framework that adds functionality for dynamically managing digital web content.

What are the web framework types?

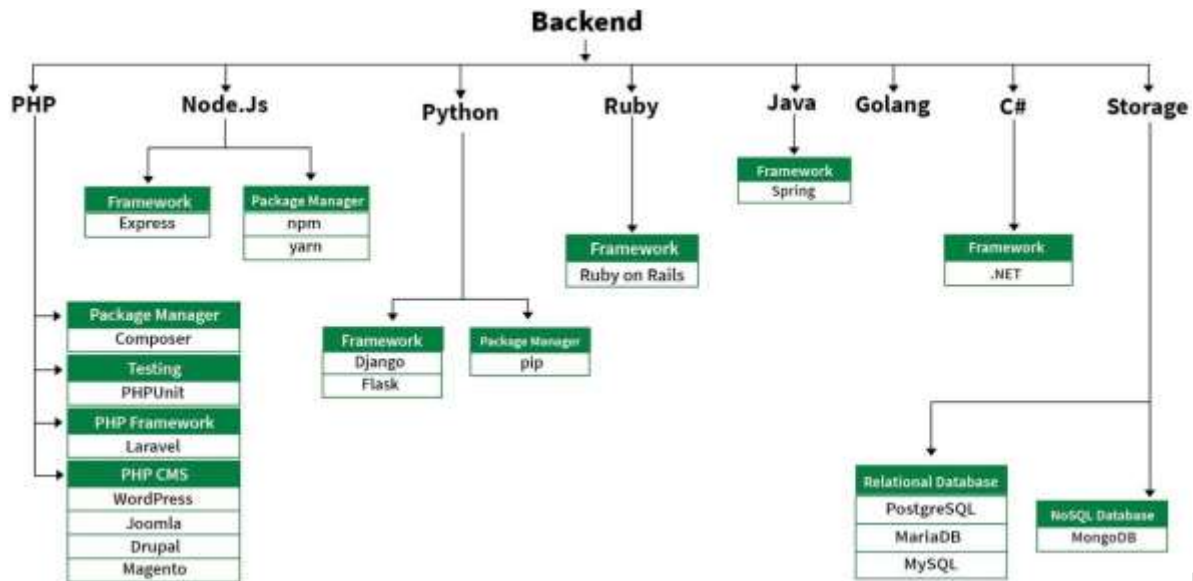
- ❖ Organizations can choose from a wide range of web development frameworks -- each offering an assortment of features -- giving development teams plenty of options from which to choose.
- ❖ Despite their differences, however, most frameworks fall into one of two categories: those that target front-end development and those that target the back end:

- ❖ **Front-end frameworks.** Also called user-side or client-side frameworks, front-end frameworks focus on the user-facing elements of a web application.
- ❖ They provide the components and templates necessary to render passive or interactive webpages in a browser, using industry technologies such as HTML, CSS, JavaScript and jQuery.



- ❖ **Back-end frameworks.** These frameworks, also called server-side frameworks, target the server and back-end components that support a web application.
- ❖ They're responsible for mapping URLs, processing HTTP requests, interfacing with data sources and supporting other back-end operations.
- ❖ Back- end frameworks use industry technologies such as Python, PHP, .NET, Java and Ruby.

SANCTIFIED



Web development frameworks are also distinguished from each other by their approach to application architecture. Many web development frameworks are based on a Model-View-Controller (MVC) architecture, which separates the web application into three layers.

- ❖ The *Model* layer is concerned with the back-end business logic and data.
- ❖ The *View* layer focuses on the user interface and facilitating interactivity.
- ❖ The *Controller* layer acts as an interface between the model and view layers, processing the requests between them.

Components of Web Development:

- Application templates for presenting information within a browser.
- Programming environment for scripting the flow of information.
- APIs for accessing back-end data resources
- Code libraries with prebuilt components and code snippets.
- Support for debugging, quality assurance (QA) testing and code reusability.

Benefits of Web Development Framework:

Makes the Development Process Easier

- ❖ Web frameworks provide pre-written code libraries, modules, and guidelines to developers, which can greatly accelerate the creation process.
- ❖ Assuring scalability, maintainability, and adherence to industry norms for the code also helps.

Eases Debugging and Application Maintenance

- ❖ Web frameworks give programmers access to tools that simplify managing and debugging their web applications.
- ❖ Frameworks frequently come with built-in debugging tools and can identify and correct **typing errors and bugs**.

Reduces Code Length

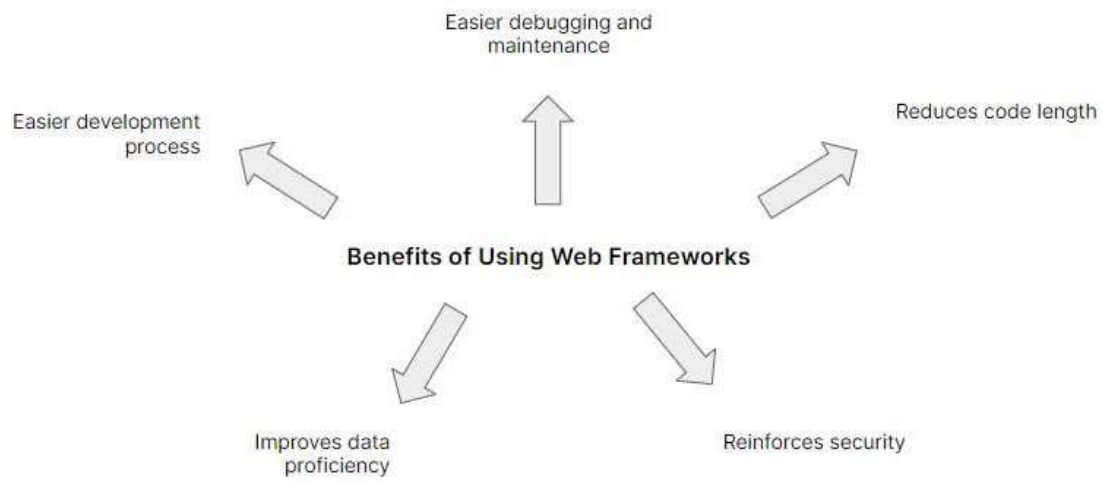
- ❖ With the aid of web frameworks, developers can achieve more with less code. Pre-built libraries and modules can speed up development by saving developers time and lowering the amount of code they need to create.

Improves Database Proficiency

- ❖ Web frameworks often include built-in tools for database integration, making it easier to work with databases.
- ❖ This can help improve database proficiency, which is essential for web applications that rely on data.

Reinforces Security

- ❖ Web frameworks provide developers with built-in security features and guidelines to help reinforce security best practices.
- ❖ This helps ensure that web applications are secure and less vulnerable to attacks.



USER:

Web Application

- ❖ A web application is software that runs in your web browser. Businesses have to exchange information and deliver services remotely.
- ❖ They use web applications to connect with customers conveniently and securely.
- ❖ The most common website features like shopping carts, product search and filtering, instant messaging, and social media newsfeeds are web applications in their design.
- ❖ They allow you to access complex functionality without installing or configuring software.

Benefits of web applications

- ❖ Web applications have several benefits, with almost all major enterprises utilizing them as part of their user offerings. Here are some of the most common benefits associated with web apps.

1. Accessibility

- Web apps can be accessed from all web browsers and across various personal and business devices.
- Teams in different locations can access shared documents, content management systems, and other business services through subscription-based web applications.

2. Efficient development

- ❖ As detailed, the development process for web apps is relatively simple and cost-effective for businesses.
- ❖ Small teams can achieve short development cycles, making web applications an efficient and affordable method of building computer programs.
- ❖ In addition, because the same version works across all modern browsers and devices, you won't have to create several different iterations for multiple platforms.

3. User simplicity

- ❖ Web apps don't require users to download them, making them easy to access while eliminating the need for end-user maintenance and hard drive capacity.
- ❖ Web applications automatically receive software and security updates, meaning they are always up to date and less at risk of security breaches.

SNVCEET

4. Scalability

Businesses using web apps can add users as and when they need, without additional infrastructure or costly hardware. In addition, the vast majority of web application data is stored in the cloud, meaning your business won't have to invest in additional storage capacity to run web apps.

Some common web applications

There are numerous types of web applications. Here are some of the most well-known.

- **Workplace collaboration web applications**

Workplace collaboration web apps allow team members to access documents, shared calendars, business instant messaging services, and other enterprise tools.

- ❖ **Ecommerce web applications**

Ecommerce web apps such as Amazon.com enable users to browse, search, and pay for products online.

- ❖ **Email web applications**

Webmail apps are widely used by enterprises and personal users to access their emails. They often include other communication tools such as instant messaging and video meetings.

- ❖ **Online banking web applications**

Business and personal users widely use online banking web apps to access their accounts and other financial products such as loans and mortgages.

- ❖ **Technical documentation**

You can use web applications to create and share technical documentation like user manuals, how-to guides and device specifications

Web Server

The term *web server* can refer to hardware or software, or both of them working together.

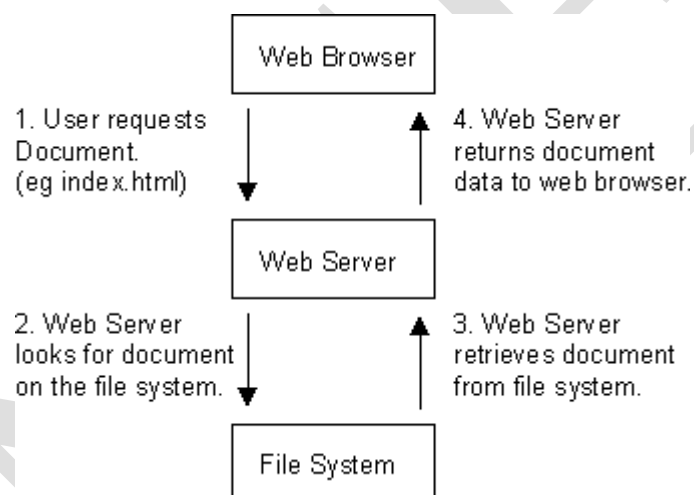
1. On the hardware side, a web server is a computer that stores web server software and a website's component files (for example, HTML documents, images, CSS stylesheets, and JavaScript files). A web server connects to the

Internet and supports physical data interchange with other devices connected to the web.

2. On the software side, a web server includes several parts that control how web users access hosted files. At a minimum, this is an *HTTP server*. An HTTP server is software that understands URLs (web addresses) and HTTP (the protocol your browser uses to view webpages). An HTTP server can be accessed through the domain names of the websites it stores, and it delivers the content of these hosted websites to the end user's device.

At the most basic level, whenever a browser needs a file that is hosted on a web server, the browser requests the file via HTTP.

When the request reaches the correct (hardware) web server, the (software) *HTTP server* accepts the request, finds the requested document, and sends it back to the browser, also through HTTP. (If the server doesn't find the requested document, it returns a 404 response instead.)



To publish a website, you need either a static or a dynamic web server.

A **static web server**, or stack, consists of a computer (hardware) with an HTTP server (software). We call it "static" because the server sends its hosted files as-is to your browser.

A **dynamic web server** consists of a static web server plus extra software, most commonly an *application server* and a *database*. We call it "dynamic" because the application server updates the hosted files before sending content to your browser via the HTTP server.

Web Browser

A software application used to access information on the World Wide Web is called a Web Browser. When a user requests some information, the web browser fetches the data from a web server and then displays the webpage on the

user's screen.

Types of Web Browser

The functions of all web browsers are the same. Thus, more than the different types there are different web browsers which have been used over the years.

Discussed below are different web browser examples and their specific features:

1. WorldWideWeb

- The first web browser ever
- Launched in 1990
- It was later named “Nexus” to avoid any confusion with the World Wide Web
- Had the very basic features and less interactive in terms of graphical interface
- Did not have the feature of bookmark

2. Mosaic

- It was launched in 1993
- The second web browser which was launched
- Had a better graphical interface. Images, text and graphics could all be integrated
- It was developed at the National Center for Supercomputing Applications
- The team which was responsible for creating Mosaic was lead by Marc Andreessen
- It was named “the world’s first popular browser”

3. Netscape Navigator

- It was released in 1994
- In the 1990s, it was the dominant browser in terms of usage share
- More versions of this browser were launched by Netscape
- It had an advanced licensing scheme and allowed free usage for non- commercial purposes

4. Internet Explorer

- It was launched in 1995 by Microsoft
- By 2003, it has attained almost 95% of usage share and had become the most popular browsers of all
- Close to 10 versions of Internet Explorer were released by Microsoft and were updated gradually

- It was included in the Microsoft Windows operating system
- In 2015, it was replaced with “Microsoft Edge”, as it became the default browser on Windows 10

5. Firefox

- It was introduced in 2002 and was developed by Mozilla Foundation
- Firefox overtook the usage share from Internet Explorer and became the dominant browser during 2003-04
- Location-aware browsing was made available with Firefox
- This browser was also made available for mobile phones, tablets, etc.

6. Google Chrome

- It was launched in 2008 by Google
- It is a cross-platform web browser
- Multiple features from old browsers were amalgamated to form better and newer features
- To save computers from malware, Google developed the ad-blocking feature to keep the user data safe and secure
- Incognito mode is provided where private searching is available where no cookies or history is saved
- Till date, it has the best user interface

Apart from these, Opera Mini web browser was introduced in 2005 which was specially designed for mobile users. Before the mobile version, the computer version “Opera” was also released in 1995. It supported a decent user interface and was developed by Opera Software.

S. No.	Parameters	Web Browser	Web Server
1.	Basics	Web Browser is an Application program that displays a World wide web document. It usually uses the internet service to access the document.	A web server is a program or the computer that provide services to other programs called client.
2.	Function	The Web browser requests the server for the web documents and services.	The Web server accepts, approves, and responds to the request made by the web browser for a web document or service.

3.	Responsibility	A web browser is a programme that uses websites to search the internet for information.	The web server is responsible for connecting websites and web browsers.
4.	Interface	The web browser acts as an interface between the server and the client and displays a web document to the client.	The web server is a software or a system which maintain the web applications, generate response and accept clients data.
5.	Components of architecture	Components of web browser architecture- a controller, client program, and interpreters.	Components of web server architecture- hardware, operating system software, and Web server software.
6.	HTTP request and response	The web browser sends an HTTP request and gets an HTTP response.	The web server gets HTTP requests and sends HTTP responses.
7.	Processing Model	Doesn't exist any processing model for the web browser.	There exist three types of processing models for web servers i.e Process-based, Thread based, and Hybrid.
8.	Storing data	Web browser stores the cookies for different websites.	Web servers provide an area to
9.	Installation	The web browser is installed on the client's computer.	The web server can be a remote machine placed on the other side of your network or even on the other end of the globe, or it is your very own personal computer at home.
10.	Examples	Examples of Web browsers are Mozilla Firefox, Google Chrome, and Internet Explorer.	An example of a Web Server is Apache Server.

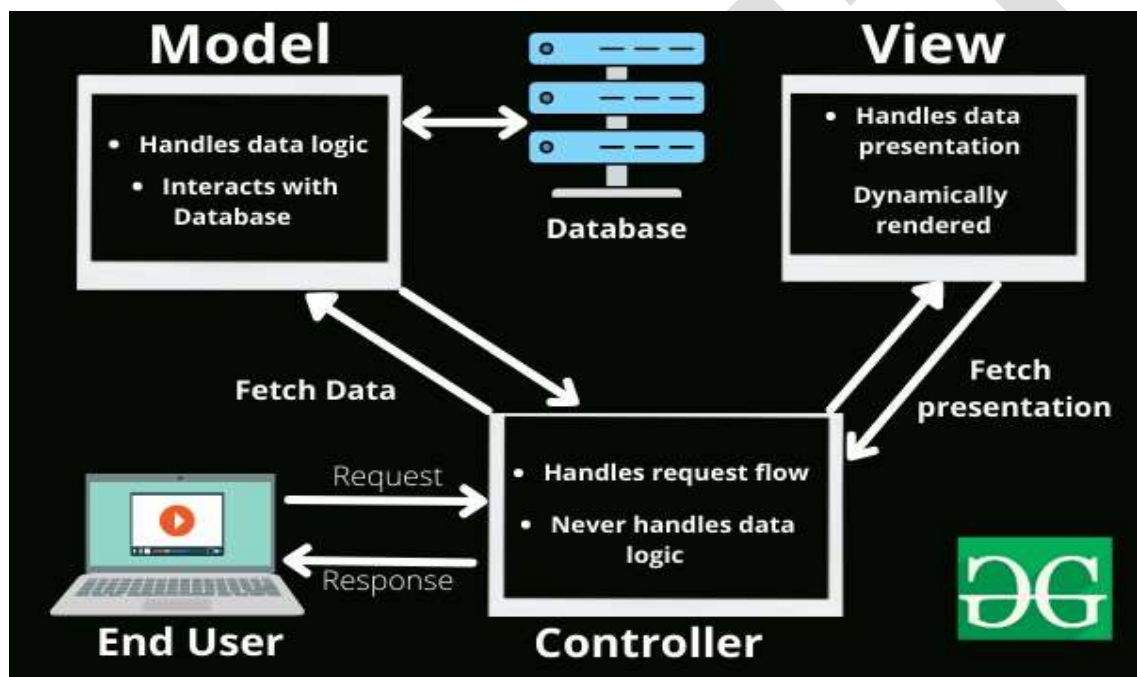
MVC Architecture:

MVC is an architectural pattern that is used to divide the application into

three components, namely - Model, View, and Controller. Separating the application into these three components makes it easier to scale the application and makes it more extensive and easier to maintain.

Features of MVC :

- It provides a clear separation of **business logic, UI logic, and input logic**.
- It offers full control over your HTML and URLs which makes it easy to design web application architecture.
- It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
- It supports **Test Driven Development (TDD)**.



Components of MVC :

The MVC framework includes the following 3 components:

Controller:

- The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary. The controller doesn't have to worry about handling data logic, it just tells the model what to do. It process all the business logic and incoming requests, manipulate data using the **Model** component and interact with the **View** to render the final output.

View:

- The **View** component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller. It only interacts with the controller.

Model:

- The **Model** component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data. It can add or retrieve data from the database. It responds to the controller's request because the controller can't interact with the database by itself. The model
- interacts with the database and gives the required data back to the controller.

Advantages of MVC:

- Codes are easy to maintain and they can be extended easily.
- The MVC **model** component can be tested separately.
- The components of MVC can be developed simultaneously.
- It reduces complexity by dividing an application into three units.

Model, view, and controller.

- It supports **Test Driven Development (TDD)**.
- It works well for Web apps that are supported by large teams of web designers and developers.
- This architecture helps to test components independently as all classes and objects are independent of each other
- **Search Engine Optimization (SEO) Friendly.**

Disadvantages of MVC:

- It is difficult to read, change, test, and reuse this model
- It is not suitable for building small applications.
- The inefficiency of data access in view.
- The framework navigation can be complex as it introduces new layers of abstraction which requires users to adapt to the decomposition criteria of MVC.
- Increased complexity and Inefficiency of data

Popular MVC Frameworks:

Some of the most popular and extensively used MVC frameworks are listed below.

- Ruby on Rails
- Django
- CherryPy
- Spring MVC
- Catalyst
- Rails
- Zend Framework
- Fuel PHP
- Laravel
- Symphony

Understanding of Different Stack: Express

- ❖ The Express module acts as the webserver in the Node.js-to-Angular stack. The fact that it is running in Node.js makes it easy to configure, implement, and control. The Express module extends Node.js to provide several key components for handling web requests.
- ❖ This allows you to implement a running webserver in Node.js with only a few lines of code. For example, the Express module provides the ability to easily set up destination routes (URLs) for users to connect to.
- ❖ It also provides great functionality on working with the HTTP request and response objects, including things like cookies and HTTP headers. The following is a partial list of the valuable features of Express:
 - Route management: Express makes it easy to define routes (URL endpoints) that tie directly to Node.js script functionality on the server.
 - Error handling: Express provides built-in error handling for documents not found and other errors.
 - Easy integration: An Express server can easily be implemented behind an existing reverse proxy system such as Nginx or Varnish. This allows it to be easily integrated into your existing secured system.
 - Cookies: Express provides easy cookie management.
 - Session and cache management: Express also enables session management and cache management.

Angular

- ❖ Angular is a client-side framework developed by Google. Angular provides all the functionality needed to handle user input in the browser, manipulate data on the client side, and control how elements are displayed in the browser view. It is written using TypeScript.
- ❖ The entire theory behind Angular is to provide a framework that makes it easy to implement web applications using the MVC framework.
- ❖ Other JavaScript frameworks could be used with the Node.js platform, such as Backbone, Ember, and Meteor.
- ❖ However, Angular has the best design, feature set, and trajectory at this writing.

Here are some of the benefits of Angular:

- Data binding: Angular has a clean method to bind data to HTML elements using its powerful scope mechanism.
- Extensibility: The Angular architecture allows you to easily extend almost every aspect of the language to provide your own custom implementations.
- Clean: Angular forces you to write clean, logical code.
- Reusable code: The combination of extensibility and clean code makes it easy to write reusable code in Angular. In fact, the language often forces you to do so when creating custom services.

- **Support:** Google is investing a lot into this project, which gives it an advantage over other similar initiatives.
- **Compatibility:** Angular is based on TypeScript, which makes it easier to begin integrating Angular into your environment and to reuse pieces of your existing code within the structure of the Angular framework.

Node.js

- i. Node.js is a development framework based on Google's V8 JavaScript engine. Therefore, Node.js code is written in JavaScript and then compiled into machine code by V8 to be executed.
- ii. Many of your backend services can be written in Node.js, as can the server-side scripts and any supporting web application functionality. The nice thing about Node.js is that it is all just JavaScript, so you can easily take functionality from a client-side script and place it in a server-side script.
- iii. Also, the web server can run directly within the Node.js platform as a Node.js module, so it makes it much easier than, say, Apache at wiring up new services or server-side scripts.
- iv. The following are just a few reasons why Node.js is a great framework to start from:
 - **JavaScript end-to-end:** One of the biggest advantages to Node.js is that it allows you to write both server- and client-side scripts in JavaScript. There have always been difficulties in deciding where to put scripting logic. Too much on the client side makes the client cumbersome and unwieldy, but too much on the server side slows down web applications and puts a heavy burden on the webserver. With Node.js you can take JavaScript written on the client and easily adapt it for the server and vice versa. Also, your client developers and server developers will be speaking the same language.
 - **Event-driven scalability:** Node.js applies a different logic to handling web requests. Rather than having multiple threads waiting to process web requests, they are processed on the same thread using a basic event model. This allows Node.js web servers to scale in ways that traditional web servers never can. This is discussed in more detail in later chapters.
 - **Extensibility:** Node.js has a great following and an active development community. New modules to extend Node.js functionality are being developed all

the time. Also it is simple to install and include new modules in Node.js, making it easy to extend a Node.js project to include new functionality in minutes.

■ Time: Let's face it, time is valuable. Node.js is super easy to set up and develop in. In only a few minutes, you can install Node.js and have a working webserver.

MongoDB

- MongoDB is an agile and scalable NoSQL database. The name Mongo comes from “humongous.”
 - It is based on the NoSQL document store model, meaning that data is stored in the database as a form of JSON objects rather than the traditional columns and rows of a relational database.
 - MongoDB provides great website backend storage for high traffic websites that need to store data such as user comments, blogs, or other items because it is fast, scalable, and easy to implement.
 - This book covers using the MongoDB driver library to access MongoDB from Node.js. Node.js supports a variety of DB access drivers, so the data store could just as easily be MySQL or some other database. However, the following are some of the reasons that MongoDB really fits in the Node.js stack well:
 - Document orientation: Because MongoDB is document-oriented, the data is stored in the database in a format close to what you will be dealing with in both server-side and clientside scripts. This eliminates the need to transfer data from rows to objects and back.
- Components 13
- High performance: MongoDB is one of the highest performing databases available. Especially today when more and more people interact with websites, it is important to have a backend that can support heavy traffic.
 - High availability: MongoDB's replication model makes it easy to maintain scalability while keeping high performance.
 - High scalability: MongoDB's structure makes it easy to scale horizontally by sharing the data across multiple servers.

- No SQL injection: MongoDB is not susceptible to SQL injection (putting SQL statements in web forms or other input from the browser that compromises the DB security) because objects are stored as objects, not using SQL strings.

React

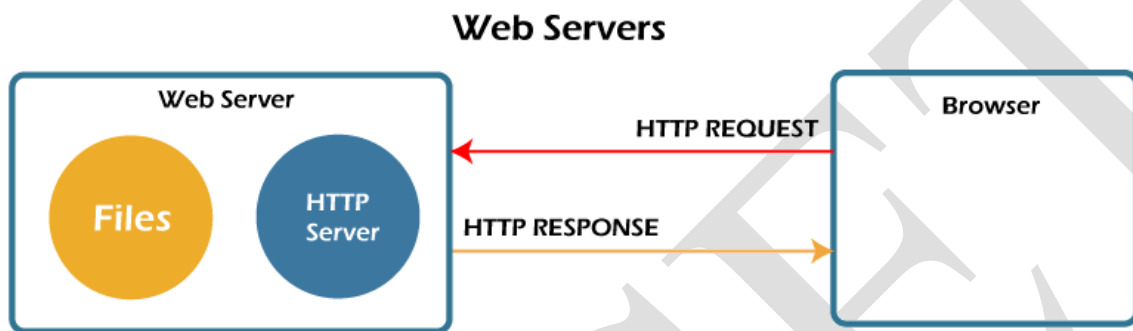
- React anchors the MERN stack. In some sense, it is the defining component of the MERN stack.
 - React is an open-source JavaScript library maintained by Facebook that can be used for creating views rendered in HTML.
 - Unlike AngularJS, React is not a framework. It is a library. Thus, it does not, by itself, dictate a framework pattern such as the MVC pattern.
-

FULL STACK WEB DEVELOPMENT –

UNIT 2

SERVER SIDE PROGRAMMING WITH NODE JS

Web Servers



- Web pages are a collection of data, including images, text files, hyperlinks, database files etc., all located on some computer (also known as server space) on the Internet.
- A web server is dedicated software that runs on the server-side. When any user requests their web browser to run any web page, the webserver places all the data materials together into an organized web page and forwards them back to the web browser with the help of the Internet.

Therefore, we can conclude that: -

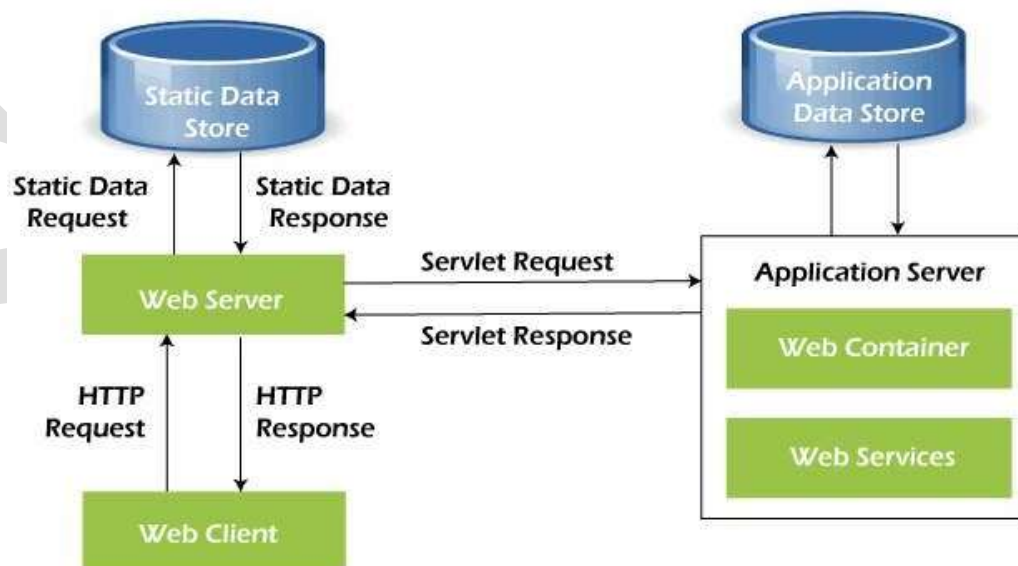
- A web server is a dedicated computer responsible for running websites sitting out on those computers somewhere on the Internet. They are specialized programs that circulate web pages as summoned by the user. The primary objective of any web server is to collect, process and provide web pages to the users.
- This intercommunication of a web server with a web browser is done with the help of a protocol named **HTTP (Hypertext Transfer Protocol)**.
- These stored web pages mostly use static content, containing HTML documents, images, style sheets, text files, etc.
- However, **web servers can serve static as well as dynamic contents**. Web Servers also assists in emailing services and storing files.

- Therefore it also uses SMTP (Simple Mail Transfer Protocol) and FTP (File Transfer Protocol) protocols to support the respective services.
- Web servers are mainly used in web hosting or hosting the website's data and running web-based applications.
- The hardware of the web servers are connected to the Internet that manages the data exchange facility within different connected devices.
- In contrast, the software of web server software is responsible for controlling how a user accesses delivered files.
- Typically, web server management is an ideal example of the client/server model.
- Therefore, **it is compulsory for all computers that host websites (whether with state or dynamic web page content) to have web server software.**

How do web servers work?

The term web server can denote server hardware or server software, or in most cases, both hardware and software

Working of web servers



1. **On the hardware side**, a web server is defined as a computer that stores

software and another website raw data, such as HTML files, images, text documents, and JavaScript files. The hardware of the web servers are connected to the web and supports the data exchange with different devices connected to the Internet.

2. **On the software side**, a web server includes server software accessed through website domain names. It controls how web users access the web files and ensures the supply of website content to the end-user. The web server contains several components, including an HTTP server

Whenever any web browser, such as Google Chrome, Microsoft Edge or Firefox, requests for a web page hosted on a web server, the browser will process the request forward with the help of HTTP. At the server end, when it receives the request, the HTTP

server will accept the request and immediately start looking for the requested data and forwards it back to the web browser via HTTP.

Let's discover the step-by-step process of what happens whenever a web browser approaches the web server and requests a web file or file. Follow the below steps:

1. First, any web user is required to **type the URL of the web page in the address bar** of your web browser.
2. With the help of the URL, your **web browser will fetch the IP address of your domain** name either by converting the URL via DNS (Domain Name System) or by looking for the IP in cache memory. The IP address will direct your browser to the web server.
3. After making the connection, the **web browser will request for the web page from the web server** with the help of an HTTP request.
4. As soon as the web server receives this request, it immediately **responds by sending back the requested page** or file to the web browser HTTP.
5. If the web page requested by the **browser does not exist or if there occurs some error in the process**, the web server will return an error message.
6. If there occurs no error, the browser will successfully display the webpage.

Examples of web server uses

Web servers are mostly used for:

- sending and receiving mails on Internet by using SMTP (Simple Mail transfer Protocol);
- fetching requests for File Transfer Protocol (FTP) files; and
- designing, developing, and publishing websites.

Many Web servers, even the basic one, also support the server-side scripting

technique. Server-side scripting is a web development method used to employ scripts on a web server that produces a customized response for each user. This technique operates on the server machine and consists of an extensive feature set, including database access. The server-side scripting process will have various scripting languages such as ASP

, PHP

, Java

, JavaScript

, Python

, ruby

and many more. This technique also enables the HTML files to be created dynamically.

Static web servers vs. Dynamic web servers

A web server can be used to store, process or deliver either static or dynamic web pages. Let's understand the definition of static and dynamic web servers with the help of their difference table given below:

S.NO	Static Web Servers	Dynamic Web Servers
.		
1	<u>Static web servers refer to the servers, which serve only the static content i.e., the content is fixed and being shown as it is.</u>	<u>Dynamic web servers refer to the servers where the content of the page can be updated and altered.</u>
2	<u>A static web server includes a computer and the HTTP (Hyper Text Transfer Protocol) software.</u>	<u>A dynamic web server also includes a computer with plenty of other software, unlike an application server and database model.</u>
3	<u>It is called static; the web pages content won't change unless the user manually changes it, and the server will deliver web files as is to the web browser.</u>	<u>It is called dynamic because the application server is used to update the web pages files at the server-side, and due to which, it can change on every call requested by the web browser.</u>
4	<u>Static web servers take less time to load the data.</u>	<u>The Dynamic web server can only produce the data when it is requested from the database. Therefore, it is time consuming and more complicated when compared to static web servers.</u>

Web server software available in the market

Though there are various web servers found in today's market, but the commonly used one are as follows:

<u>S.NO</u>	<u>Web Server Description</u>
1.	<u>Apache HTTP Server</u> <u>This web server is developed by Apache Software Foundation. It is an open-source, accessible web server available for almost all operating systems, including Windows, MACOS, Linus, FreeBSD, etc. Apache is one of the most popular web servers used around the globe.</u>
2.	<u>Microsoft Internet Information Services (IIS)</u> <u>IIS is a high-performance web server that is developed by Microsoft only for Microsoft platforms. This webs server is tightly integrated with Microsoft operating system; therefore, it is not open-sourced.</u>
3.	<u>Nginx</u> <u>Nginx is an open-source web server commonly used by administrators as it supports light resource application and scalability.</u>
4.	<u>Lighttpd</u> <u>Lighttpd, also known as lighty, is a free, open-source web server with the FreeBSD operating system. This web server is fast, secure and consumes much less CPU power. It can also run on the commonly used operating system, unlike Windows, Mac OS X, Linus.</u>
5.	<u>Sun Java System Web Server</u> <u>Sun Java is a free web server developed by Sun Microsystems well equipped for a medium and large website that can run on Windows, Linux and Unix. Moreover, this web server supports several languages, scripts and technologies essential for Web 2.0, unlike JSP, Java Servlets, PHP, Python, HTML, etc. Though Sun Java is free, it is not an open-source web server.</u>

Web server security Methods

Though there are various security techniques available in the market that a user can implement to have a safe web server experience, below given are some examples of security practices that can include processes:

1. **A reverse proxy** is a proxy server that is accessible to the clients, therefore hiding the internal server. It acts as an intermediary as wherever any user

makes requests to the web server for any data or file, the proxy server seizes those requests and then communicates with the webserver.

2. **Access restriction** is a technique that limits the web host's access to infrastructure machines or using Secure Socket Shell (SSH);
3. **Keeping web servers mended and updated**, as it benefits to ensure the web server isn't vulnerable to exposures;
4. **Network monitoring** is a security practice that ensures that no unauthorized activity is going on the web server; and
5. **Using a firewall and SSL safeguards** the web server as firewalls can supervise HTTP request traffic while a Secure Sockets Layer (SSL) supports securing the data.

JAVASCRIPT IN THE DESKTOP WITH NODEJS

Node.js Tutorial

Node.js tutorial provides basic and advanced concepts of Node.js. Our Node.js tutorial is designed for beginners and professionals both.

Node.js is a cross-platform environment and library for running JavaScript applications which is used to create networking and server-side applications.

Our Node.js tutorial includes all topics of Node.js such as Node.js installation on windows and linux, REPL, package manager, callbacks, event loop, os, path, query string, cryptography, debugger, URL, DNS, Net, UDP, process, child processes, buffers, streams, file systems, global objects, web modules etc. There are also given Node.js interview questions to help you better understand the Node.js technology.

What is Node.js

Node.js is a cross-platform runtime environment and library for running JavaScript applications outside the browser. It is used for creating server-side and networking web applications. It is open source and free to use.

Many of the basic modules of Node.js are written in JavaScript. Node.js is mostly used to run real-time server applications.

The definition given by its official documentation is as follows:

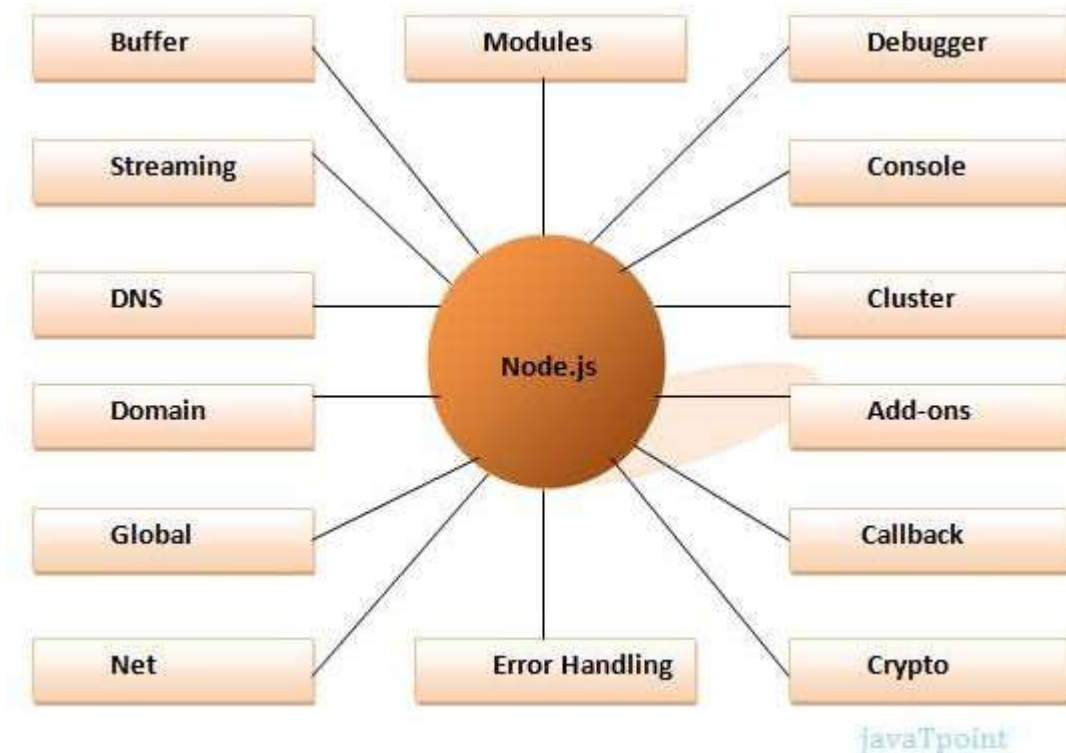
?Node.js is a platform built on Chrome's JavaScript runtime for easily building fast and scalable network applications. Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient, perfect for data-intensive real-time applications that run across distributed devices.?

Node.js also provides a rich library of various JavaScript modules to simplify the development of web applications.

1. Node.js = Runtime Environment + JavaScript Library

Different parts of Node.js

The following diagram specifies some important parts of Node.js:



Features of Node.js

Following is a list of some important features of Node.js that makes it the first choice of software architects.

1. **Extremely fast:** Node.js is built on Google Chrome's V8 JavaScript Engine, so its library is very fast in code execution.
2. **I/O is Asynchronous and Event Driven:** All APIs of Node.js library are asynchronous i.e. non-blocking. So a Node.js based server never waits for an API to return data. The server moves to the next API after calling it and a notification mechanism of Events of Node.js helps the server to get a response from the previous API call. It is also a reason that it is very fast.
3. **Single threaded:** Node.js follows a single threaded model with event looping.
4. **Highly Scalable:** Node.js is highly scalable because event mechanism helps the server to respond in a non-blocking way.
5. **No buffering:** Node.js cuts down the overall processing time while uploading audio and video files. Node.js applications never buffer any data. These applications simply output the data in chunks.
6. **Open source:** Node.js has an open source community which has produced many excellent modules to add additional capabilities to Node.js applications.
7. **License:** Node.js is released under the MIT license.

NPM

What is npm

npm is a short form of **Node Package Manager**, which is the world's largest software registry. The open-source web project developers use it from the entire world to **share** and **borrow** packages. The npm also acts as a command-line utility for the Node.js project for installing packages in the project, dependency management, and even version management.

Components of npm

npm mainly consists of three different components, these are:

1. **Website:** The npm official website is used to find packages for your project, create and set up profiles to manage and access private and public packages.
2. **Command Line Interface (CLI):** The CLI runs from your computer's terminal to interact with npm packages and repositories.
3. **Registry:** The registry is a large and public database of JavaScript projects and meta-information. You can use any supported npm registry that you want or even your own. You can even use someone else's registry as per their terms of use.

Installing npm (Node.js and npm)

npm comes with **Node.js**, which means you have to install Node.js to get installed automatically on your personal computer. There are two different

methods to install npm on a computer.

1. **Using nvm (Node Version Manager)**
2. **Using Node installer**

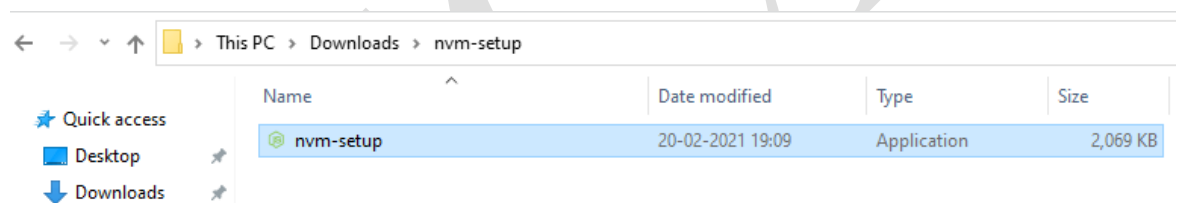
npm installation is highly recommended using nvm (Node Version Manager) rather than using Node installer. The Node installation process installs npm in the directory by allowing local permissions, and it generates an error message when you try to run npm packages globally.

1. Install npm and Node.js using nvm

It is highly recommended by npm's to use Node version manager to install npm and Node.js on your device. Installing node.js using nvm allows installing different versions of Node.js on your computer, and you can use any Node.js version for your project by switching as per your requirement. They do not recommend to us Node installers to install npm and Node.js.

To download and install npm using nvm on your **Linux operating system**, open <https://github.com/nvm-sh/nvm>.

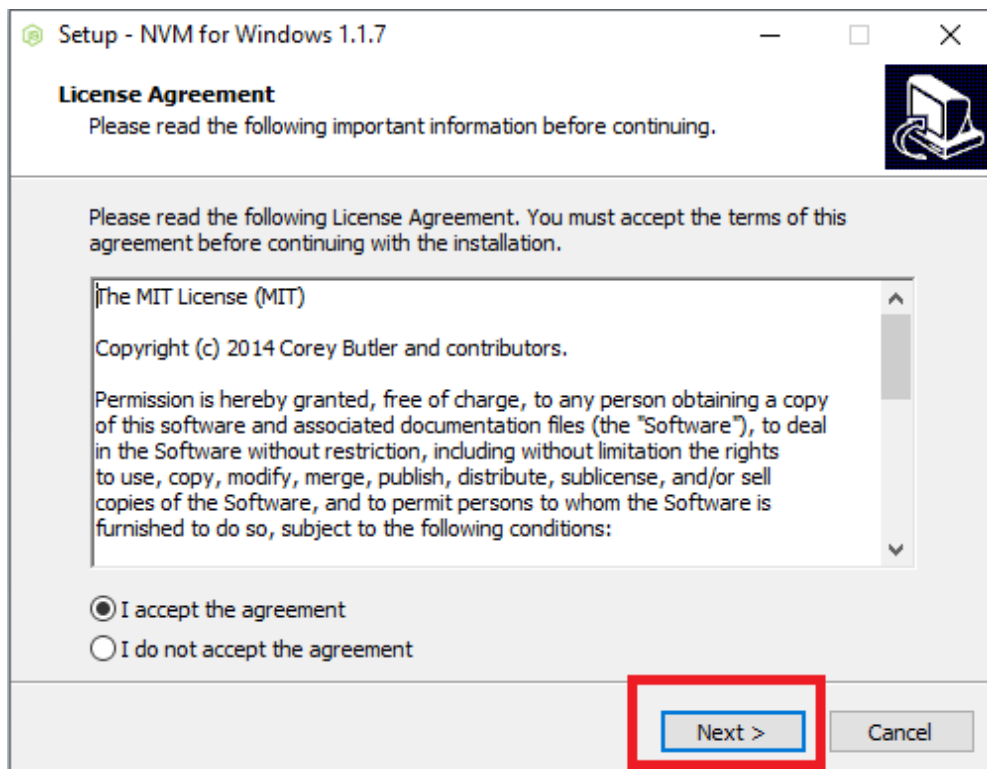
1. We will download the **nvm-setup.zip** file for Windows from Git link as we are using a Windows OS device.
2. After downloading, run the setup and follow the installation instruction



as appears on the screen.

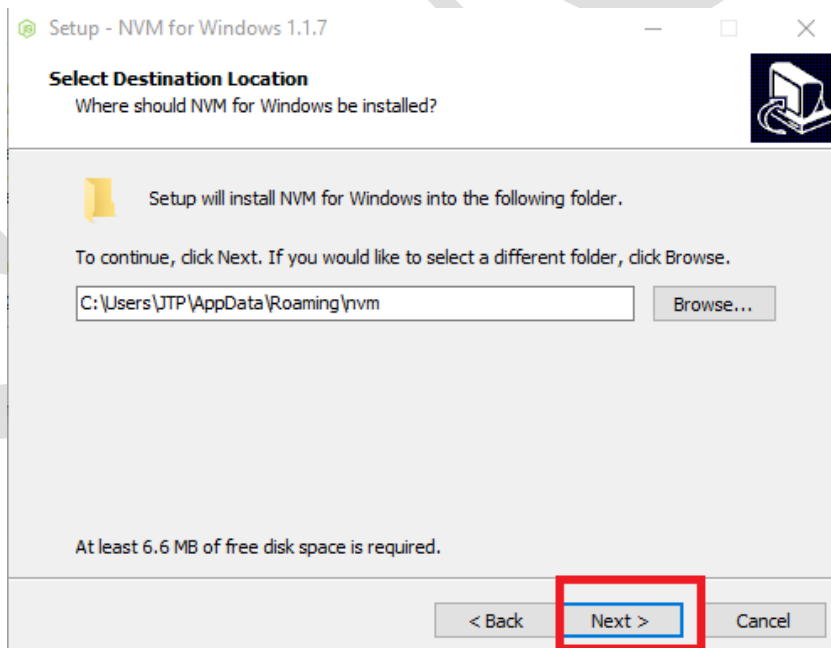
3. Run the setup either by double-clicking on it or by right-clicking and Run as administration. Allow the installation by clicking on the **Yes**

4. button. Read their license agreement carefully and if you agree with it,

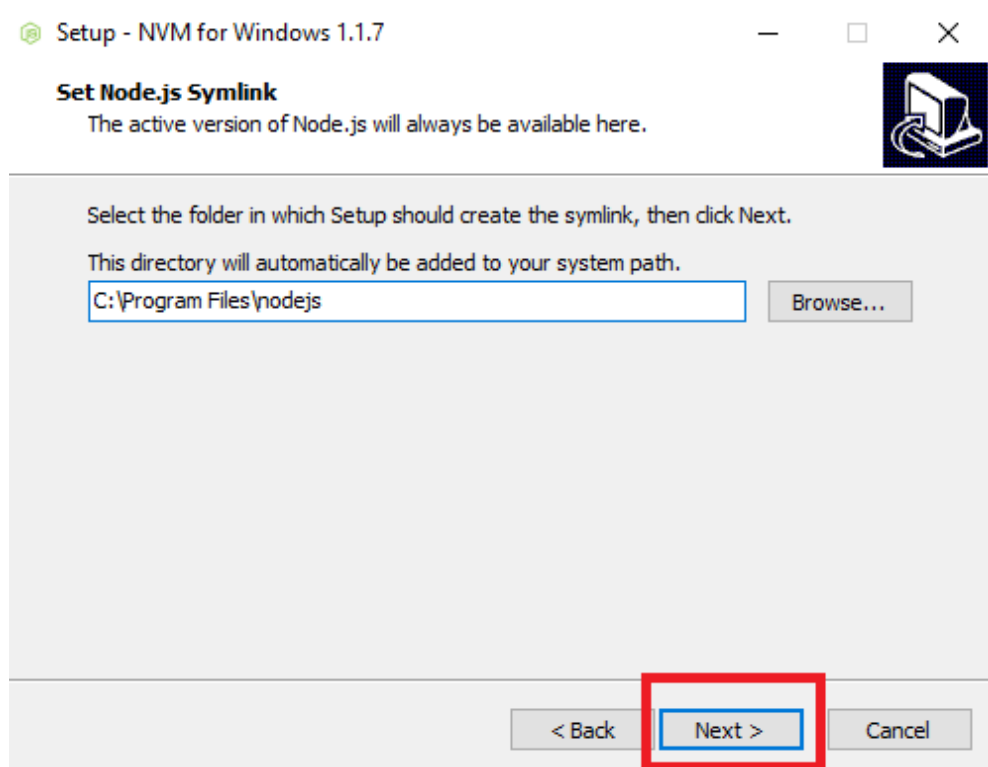


then accept it and click on **Next** button.

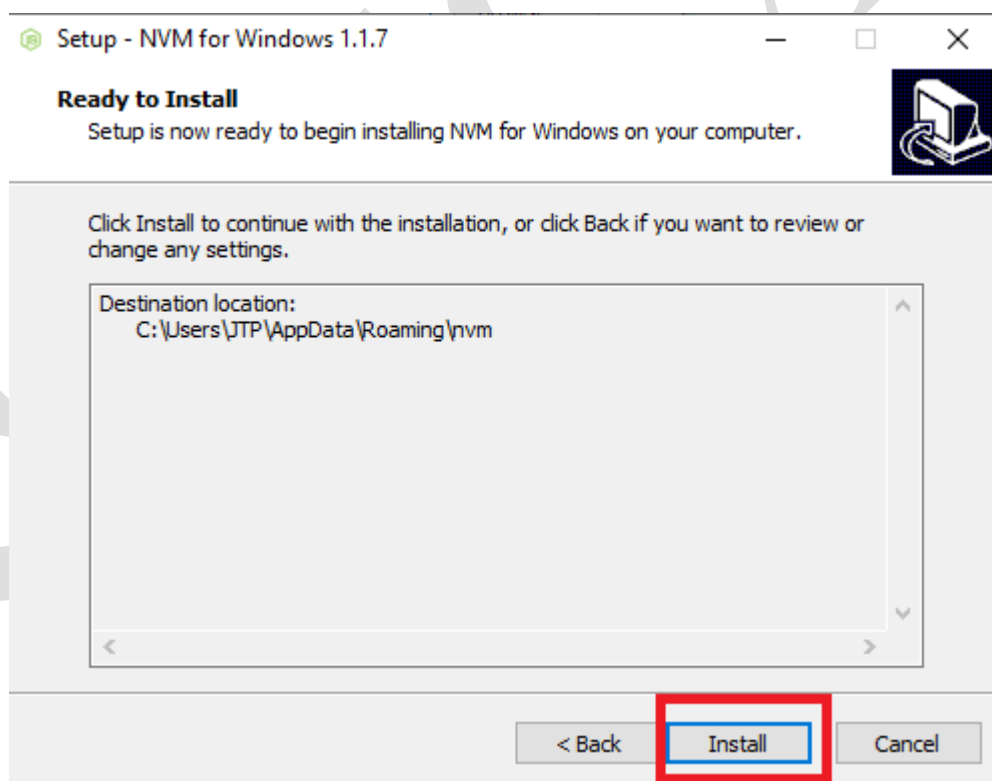
5. Select the destination location where you want to install NVM and click on the **Next** button. We leave it as the default selected location.



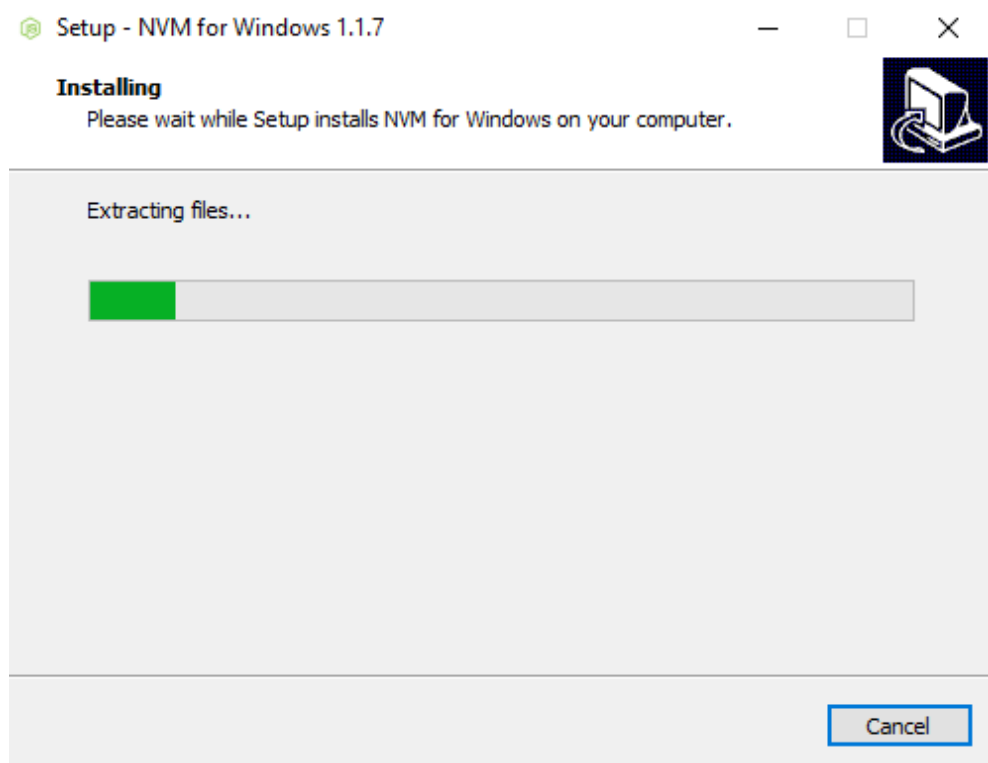
6. Select the folder location in which the setup will create the



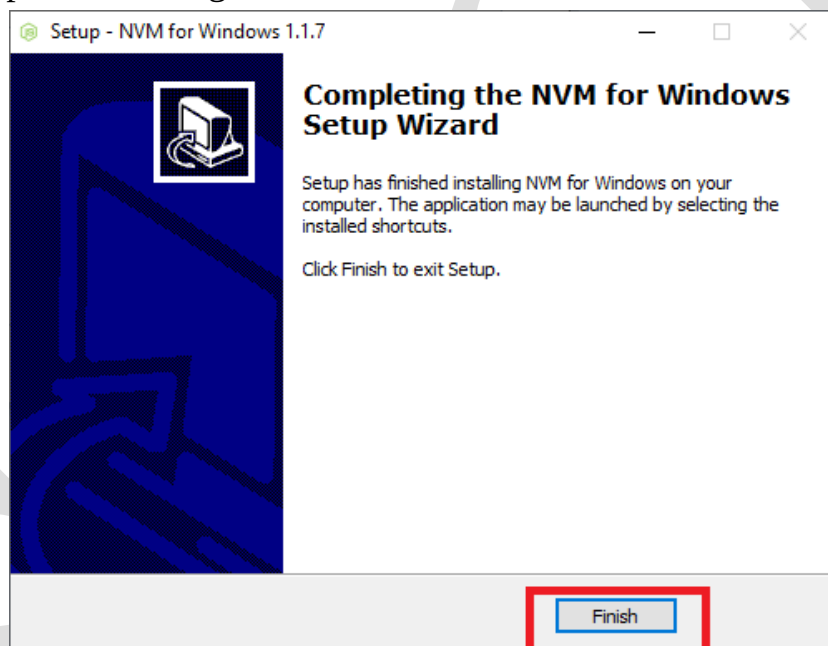
symlink and click on the **Next** button.



7. Now, your setup is ready to install, click on the **Install** button to start the installation process.
8. Now, the nvm-setup is installed on your personal computer, and it will take few seconds to wait to complete it.



our nvm setup installation gets finished; click on the **Finish** button to exit from setup.



Check nvm version

After the nvm setup gets installed, ensure whether you get installed or which version on nvm is installed. To do this, run the following command on your terminal:

```
nvm - v
```

The above command displays the nvm version running on your device and another nvm command with their short descriptions.

```
Administrator: Command Prompt
C:\>nvm -v
Running version 1.1.7.
Usage:
nvm arch : Show if node is running in 32 or 64 bit mode.
nvm install <version> [arch] : The version can be a node.js version or "latest" for the latest stable version.
                                Optionally specify whether to install the 32 or 64 bit version (defaults to system arch)
                                Set [arch] to "all" to install 32 AND 64 bit versions.
                                Add --insecure to the end of this command to bypass SSL validation of the remote download.
nvm list [available] : List the node.js installations. Type "available" at the end to see what can be installed
nvm on : Enable node.js version management.
nvm off : Disable node.js version management.
nvm proxy [url] : Set a proxy to use for downloads. Leave [url] blank to see the current proxy.
                  Set [url] to "none" to remove the proxy.
nvm node_mirror [url] : Set the node mirror. Defaults to https://nodejs.org/dist/. Leave [url] blank to use default
nvm npm_mirror [url] : Set the npm mirror. Defaults to https://github.com/npm/cli/archive/. Leave [url] blank to use default
nvm uninstall <version> : The version must be a specific version.
nvm use [version] [arch] : Switch to use the specified version. Optionally specify 32/64bit architecture.
                          nvm use <arch> will continue using the selected version, but switch to 32/64 bit mode.
nvm root [path] : Set the directory where nvm should store different versions of node.js.
                  If <path> is not set, the current root will be displayed.
nvm version : Displays the current running version of nvm for Windows. Aliased as v.
C:\>
```

You can also check the nvm version using the command: **nvm version**.

Check the list of Node.js available to install using nvm

To find the supported Node.js version to install using nvm, use the following command. It will display the partial available list of Node.js versions.

1. npm list available

```
C:\>nvm list available
```

CURRENT	LTS	OLD STABLE	OLD UNSTABLE
15.9.0	14.15.5	0.12.18	0.11.16
15.8.0	14.15.4	0.12.17	0.11.15
15.7.0	14.15.3	0.12.16	0.11.14
15.6.0	14.15.2	0.12.15	0.11.13
15.5.1	14.15.1	0.12.14	0.11.12
15.5.0	14.15.0	0.12.13	0.11.11
15.4.0	12.20.2	0.12.12	0.11.10
15.3.0	12.20.1	0.12.11	0.11.9
15.2.1	12.20.0	0.12.10	0.11.8
15.2.0	12.19.1	0.12.9	0.11.7
15.1.0	12.19.0	0.12.8	0.11.6
15.0.1	12.18.4	0.12.7	0.11.5
15.0.0	12.18.3	0.12.6	0.11.4
14.14.0	12.18.2	0.12.5	0.11.3
14.13.1	12.18.1	0.12.4	0.11.2
14.13.0	12.18.0	0.12.3	0.11.1
14.12.0	12.17.0	0.12.2	0.11.0
14.11.0	12.16.3	0.12.1	0.9.12
14.10.1	12.16.2	0.12.0	0.9.11
14.10.0	12.16.1	0.10.48	0.9.10

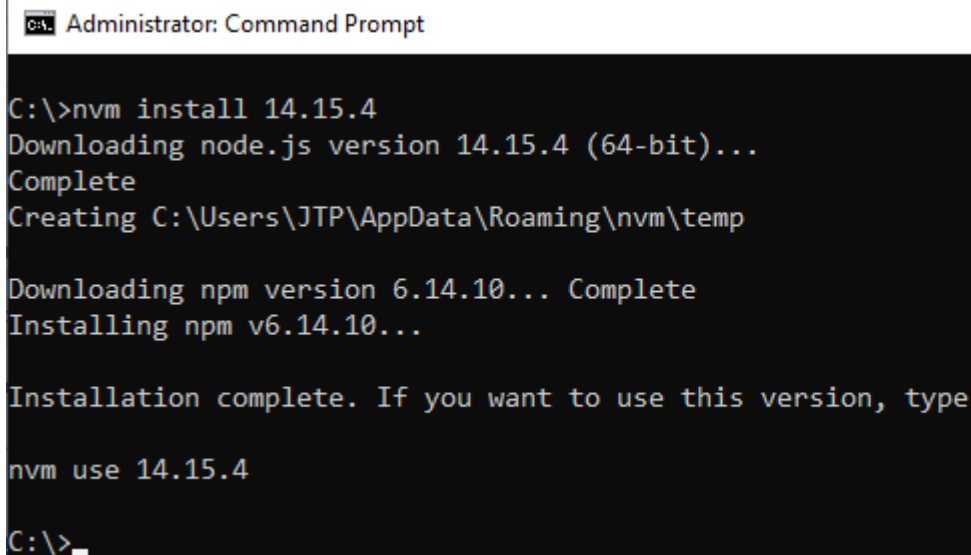
This is a partial list. For a complete list, visit <https://nodejs.org/download/release>

In the above list, **CURRENT** refers to the current version of Node.js, and **LTS** refers to the long-term support version.

Installing Node.js and npm

You can choose any version of Node.js to install; we are installing LTS 14.15.4 for our machine. Simply run the following command to install Node.js:

1. `nvm install 14.15.4`



```
Administrator: Command Prompt

C:\>nvm install 14.15.4
Downloading node.js version 14.15.4 (64-bit)...
Complete
Creating C:\Users\JTP\AppData\Roaming\nvm\temp

Downloading npm version 6.14.10... Complete
Installing npm v6.14.10...

Installation complete. If you want to use this version, type

nvm use 14.15.4

C:\>
```

This command installs Node.js (64-bit or 32-bit) and npm together depending on your device.

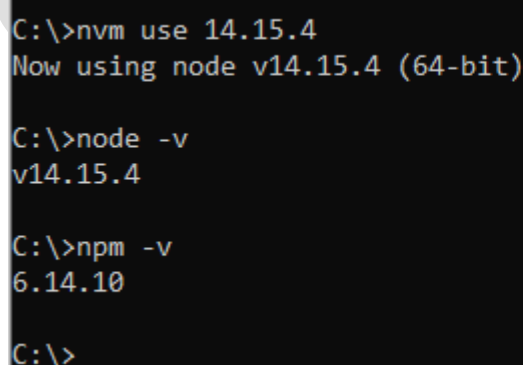
Check the version of Node.js and npm

To use Node.js use the command **nvm use 14.15.4** (make sure to replace 14.15.4 with your installed version).

1. `nvm use 14.15.4`

After that, run the command **node -v** and **npm -v** on your terminal to check the version of Node.js and npm, respectively.

1. `node -v`
2. `npm -v`



```
C:\>nvm use 14.15.4
Now using node v14.15.4 (64-bit)

C:\>node -v
v14.15.4

C:\>npm -v
6.14.10

C:\>
```

Check the list of Node.js versions installed.

You can even check the total number of Node.js versions installed and running on your computer using the below command.

1. `nvm list`

```
Administrator: Command Prompt

C:\>nvm list

* 14.15.4 (Currently using 64-bit executable)

C:\>
```

Currently, we have the Node.js version (14.15.4) running on our device. We can also be able to install more different versions of Node.js on a computer if required (while building another Node.js project based on a different version). To install another version of Node.js, choose a version from the nvm list available and run the command given below.

1. `nvm install 12.20.2`

```
C:\>nvm install 12.20.2
Downloading node.js version 12.20.2 (64-bit)...
Complete
Creating C:\Users\JTP\AppData\Roaming\nvm\temp

Downloading npm version 6.14.11... Complete
Installing npm v6.14.11...

Installation complete. If you want to use this version, type

nvm use 12.20.2
```

Here, we have selected another Node version 12.20.2 to install, and it also downloads and installs compatible npm on our device.

Now, we have two different versions of Node.js and npm. To check the total number of installed Node.js version, run the following command:

1. `nvm list`

```
C:\>nvm list

* 14.15.4 (Currently using 64-bit executable)
  12.20.2

C:\>
```

Now, we have two different versions of Node.js and npm on our computers.

2. Install npm and Node.js using Node installer

Suppose you are not able to install, or you are facing some difficulty while installing npm using Node version manager on your device. The other way to install both npm and Node.js on your device is using **the Node.js installer**.

Navigate to the Node.js download section, where you will find Node.js installer for the different operating systems to install Node.js and npm together.

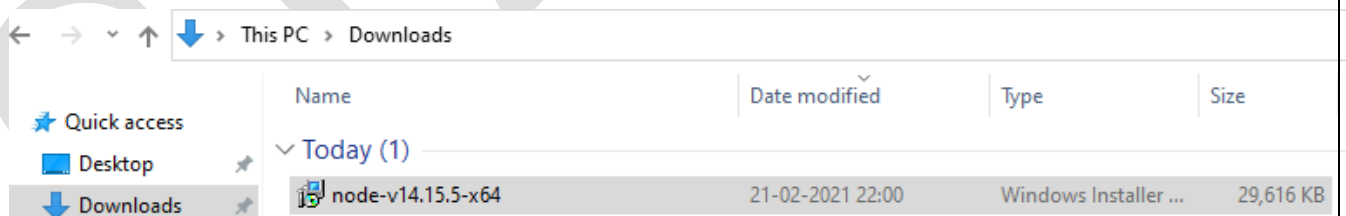


The screenshot shows the Node.js download page. The 'LTS' tab is selected, which is recommended for most users. Below the tabs, there are three main sections: 'Windows Installer' (node-v14.15.5-x64.msi), 'macOS Installer' (node-v14.15.5.pkg), and 'Source Code' (node-v14.15.5.tar.gz). On the left, there is a list of download options: Windows Installer (.msi), Windows Binary (.zip), macOS Installer (.pkg), macOS Binary (.tar.gz), Linux Binaries (x64), Linux Binaries (ARM), and Source Code. On the right, there is a table of download links for various operating systems and architectures. The '64-bit' link for Windows is highlighted with a red box.

32-bit	64-bit
32-bit	64-bit
64-bit	
64-bit	
64-bit	
ARMv7	ARMv8
node-v14.15.5.tar.gz	

Depending on your computer operating system and its bits, download any supported Node.js installer.

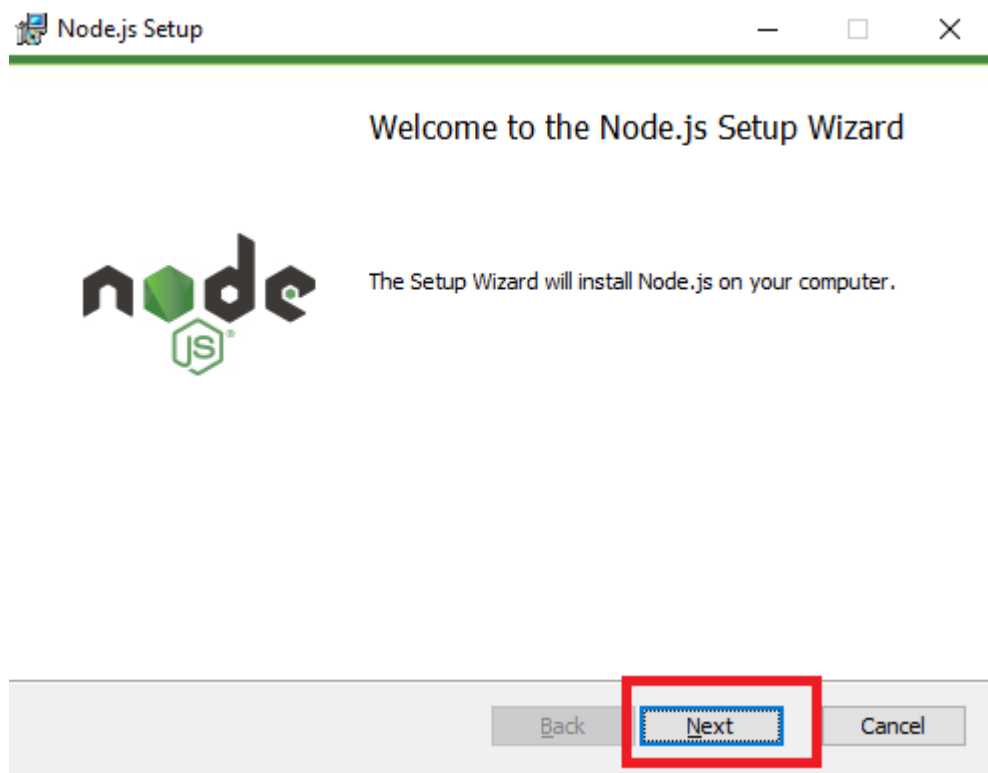
1. We have Windows with the 64-bit operating system, and we are downloading a 64-bit Node.js installer for our device.
2. After downloading, run the setup and follow the installation instruction as shown on the screen.



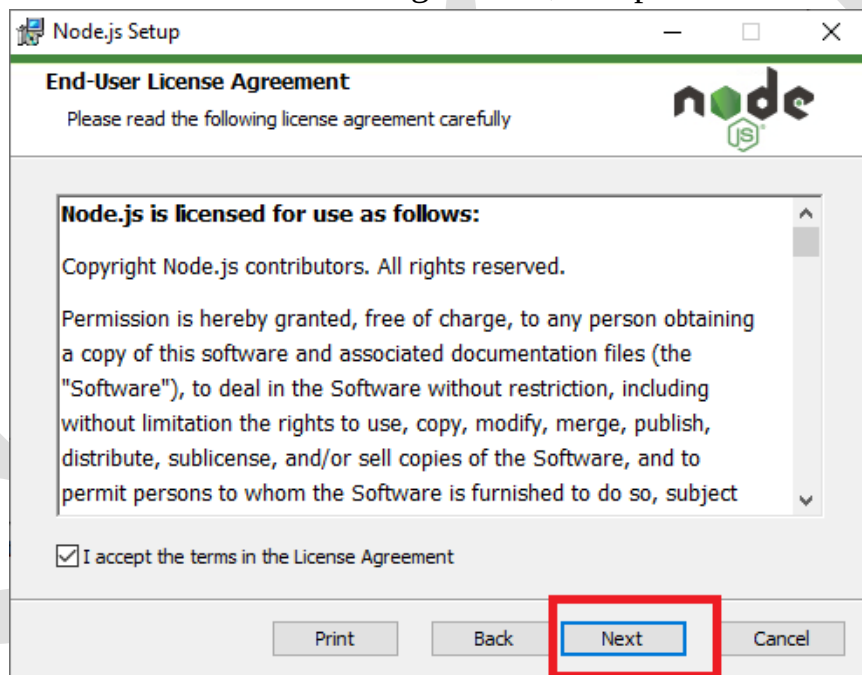
The screenshot shows a Windows File Explorer window with the address bar set to 'This PC > Downloads'. The left sidebar shows 'Quick access' with 'Desktop' and 'Downloads' highlighted. The main area displays a table of files. The file 'node-v14.15.5-x64' is listed with a size of 29,616 KB.

Name	Date modified	Type	Size
Today (1)			
node-v14.15.5-x64	21-02-2021 22:00	Windows Installer ...	29,616 KB

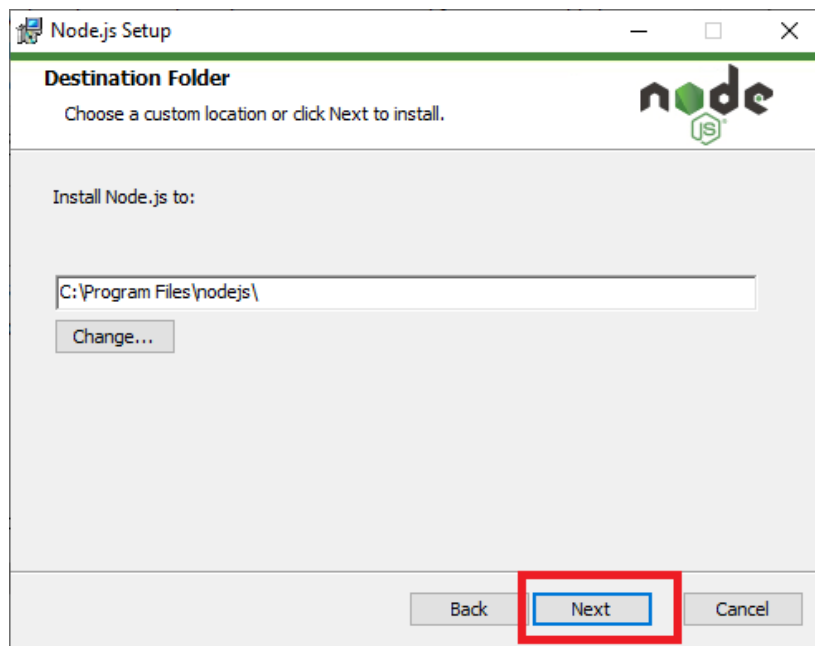
3. Click on the **Next** button to start the installation process.



3. Read the end-user license agreement, accept it and click on the **Next** button.

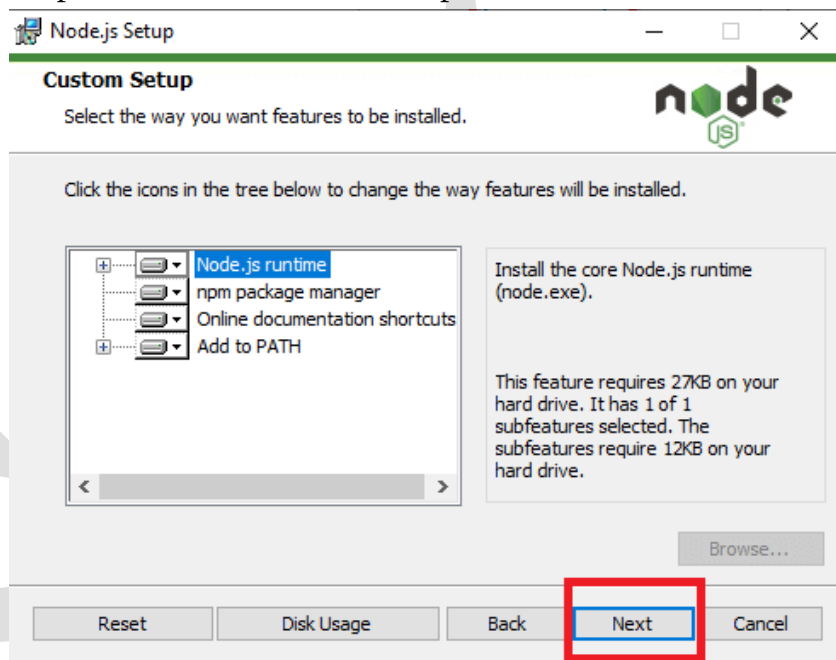


4. Choose a custom location of your Node.js and click on **Next** to install.

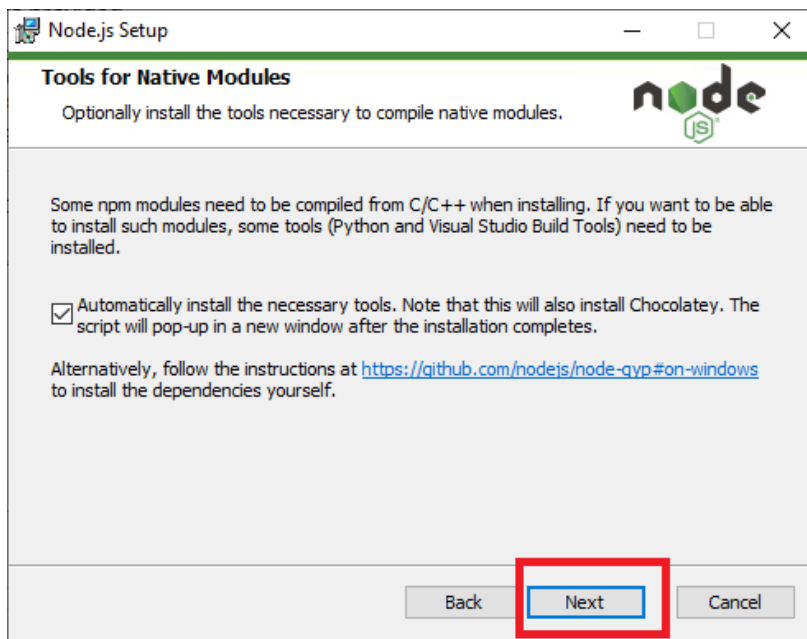


We keep it as its default location.

5. Select the feature of Node.js to get installed and click on **Next**. We keep it as its default feature provided.

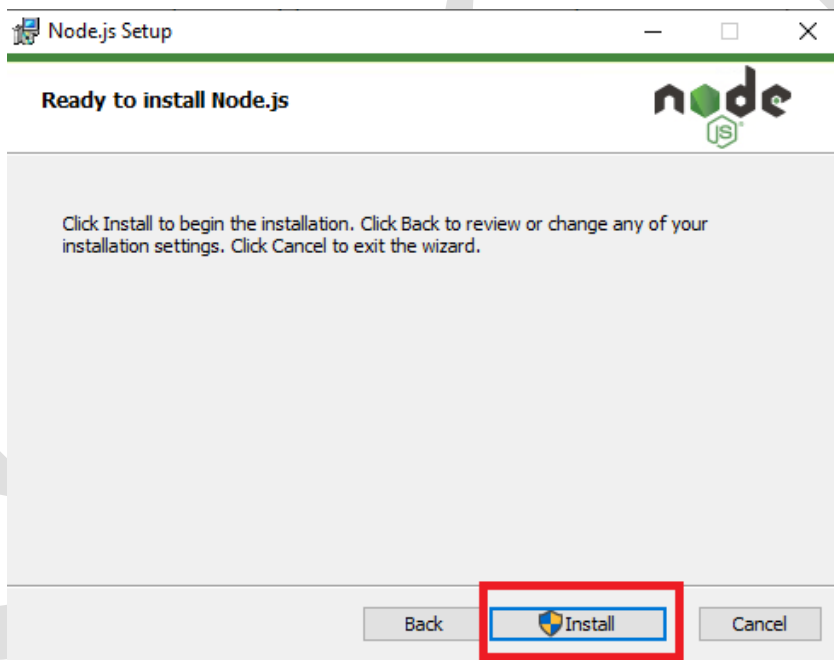


6. If you want to install some other necessary tools (C/C++, Chocolatey) with



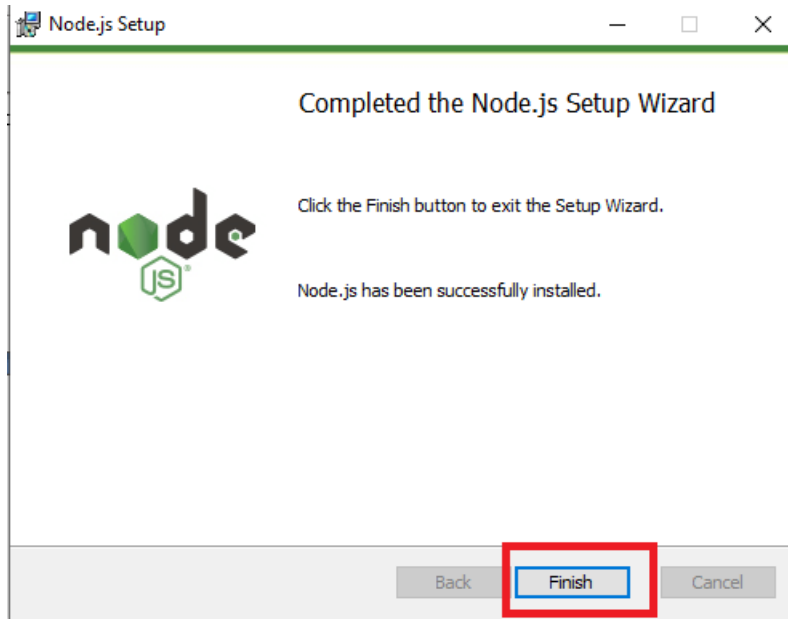
your node.js and npm, checkmark the box and click on the **Next** button.

7. Now, click on the **Install** button to install Node.js, which also installs npm on your device.

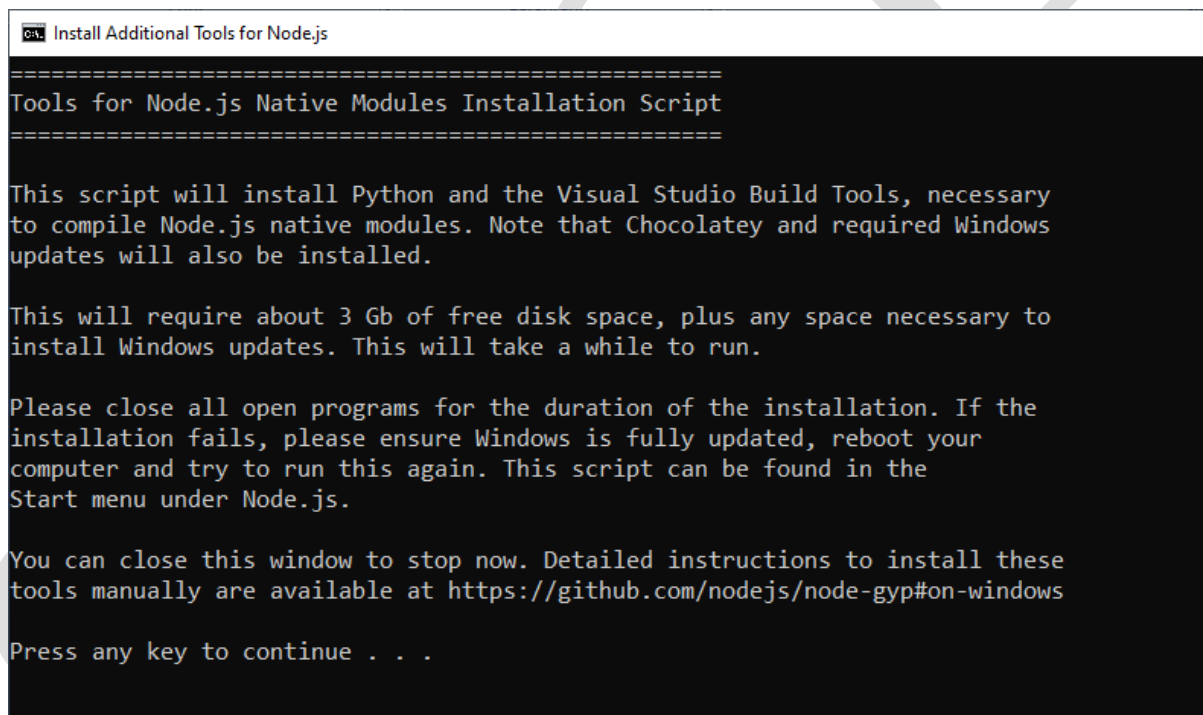


8. Click on **Yes** to install Node.js on the popup window.

9. Click on the **Finish** button to complete the installation.



The additional tools are ready to install, as we marked above to install additional tools automatically. Press any key to continue.



Run the following command on the command line to download the latest version of npm.

1. `npm install -g npm`

Properties of npm:

- All **npm** packages are defined in files called **package.json**.
- You must write the content of **package.json** in **JSON**.
- At least two fields must be present in the definition file: **name** and **version**.

package.json

package.json file will be generated when we run the **npm init** to initialize a JavaScript/Node.js project with the basic metadata information provided by developers:

name: it is the name of your JavaScript library/project

version: it is the version of your project

description: pass the project's description information

license: it is a project's license

When to use npm

- Use npm to adapt packages of code for your apps or incorporate packages as they are.
- npm can be used when you want to download standalone tools right away.
- Run packages without downloading using npx.
- Use npm when you need to share code with any npm user, anywhere.
- npm can be used when you want to restrict code to specific developers.
- Use npm to create organizations to coordinate package maintenance, coding, and developers.
- Use npm to manage multiple versions of code and code dependencies.
- Use npm to update applications easily when the underlying code is updated.
- Use npm to find other developers who are working on similar problems and projects.

SERVING FILES WITH THE HTTP MODULE

A basic necessity for most http servers is to be able to serve static files. Thankfully, it is not that hard to do in Node.js. First you read the file, then you serve the file. Here is an example of a script that will serve the files in the current directory:

```
var fs =  
  require('fs'), http  
  = require('http');  
  
http.createServer(function (req, res) {  
  fs.readFile(__dirname + req.url, function  
    (err,data) { if (err) {  
      res.writeHead(404);  
      res.end(JSON.stringify(  
        err)); return;  
    }  
  });  
});
```

```
}  
res.writeHead(2  
00);  
res.end(data);  
});  
}).listen(8080);
```

This example takes the path requested and it serves that path, relative to the local directory. This works fine as a quick solution; however, there are a few problems with this approach. First, this code does not correctly handle mime types. Additionally, a proper static file server should really be taking advantage of client side caching, and should send a "Not Modified" response if nothing has changed. Furthermore, there are security bugs that can enable a malicious user to break out of the current directory. (for example, GET ../../../../).

Each of these can be addressed individually without much difficulty. You can send the proper mime type header. You can figure how to utilize the client caches. You can take advantage of `path.normalize` to make sure that requests don't break out of the current directory. But why write all that code when you can just use someone else's library?

There is a good static file server called `node-static` written by Alexis Sellier which you can leverage. Here is a script which functions similarly to the previous one:

```
var static = require('node-  
static'); var http =  
require('http');  
  
var file = new(static.Server)(__dirname);  
  
http.createServer(function (req, res) {  
  file.serve(req, res);  
}).listen(8080);
```

This is a fully functional file server that doesn't have any of the bugs previously mentioned. This is just the most basic set up, there are more things you can do if you look at the api. Also since it is an open source project, you can always

modify it to your needs (and feel free to contribute back to the project.

INTRODUCTION TO EXPRESS FRAMEWORK

Node.js - Express Framework

Express Overview

Express is a minimal and flexible Node.js web application framework that provides a robust set of features to develop web and mobile applications. It facilitates the rapid development of Node based Web applications. Following are some of the core features of Express framework

—

- Allows to set up middlewares to respond to HTTP Requests.
- Defines a routing table which is used to perform different actions based on HTTP Method and URL.
- Allows to dynamically render HTML Pages based on passing arguments

to templates. Installing Express

Firstly, install the Express framework globally using NPM so that it can be used to create a web application using node terminal.

```
$ npm install express --save
```

The above command saves the installation locally in the **node_modules** directory and creates a directory **express** inside **node_modules**. You should install the following important modules along with **express** —

- **body-parser** — This is a node.js middleware for handling JSON, Raw, Text and URL encoded form data.
- **cookie-parser** — Parse Cookie header and populate **req.cookies** with an object keyed by the cookie names.
- **multer** — This is a node.js middleware for handling multipart/form-data.

```
$ npm install body-parser --save
```

```
$ npm install cookie-parser --save
```

```
$ npm install multer --save
```

Hello world Example

Following is a very basic Express app which starts a server and listens on port 8081 for connection. This app responds with **Hello World!** for requests to the homepage. For every other path, it will respond with a **404 Not Found**.

```
var express =  
require('express'); var app  
= express();
```

```
app.get('/', function (req,  
  res) { res.send('Hello  
  World');  
})
```

```
var server = app.listen(8081, function
```

```
() { var host =  
server.address().address  
var port = server.address().port  
  
console.log("Example app listening at http://%s:%s", host, port)  
})
```

Save the above code in a file named server.js and run it with the following command.

```
$ node server.js
```

You will see the following output –

Example app listening at

http://0.0.0.0:8081

Open <http://127.0.0.1:8081/> in any browser to see the following result.



Request & Response

Express application uses a callback function whose parameters are **request** and **response** objects.

```
app.get('/', function (req, res) {  
  // --  
})
```

- Request Object – The request object represents the HTTP request and has properties for the request query string, parameters, body, HTTP headers, and so on.
- Response Object – The response object represents the HTTP response that an Express app sends when it gets an HTTP request.

You can print **req** and **res** objects which provide a lot of information related to HTTP request and response including cookies, sessions, URL, etc.

Basic Routing

We have seen a basic application which serves HTTP request for the homepage. Routing refers to determining how an application responds to a client request to a particular endpoint, which is a URI (or path) and a specific HTTP request method (GET, POST, and so on).

We will extend our Hello World program to handle more types of HTTP requests.

```
var express = require('express');
var app = express();
```

```
// This responds with "Hello World" on the
homepage app.get('/', function (req, res) {
  console.log("Got a GET request for the
  homepage"); res.send('Hello GET');
})
```

```
// This responds a POST request for the homepage
app.post('/', function (req, res) {
  console.log("Got a POST request for the
  homepage"); res.send('Hello POST');
})
```

```
// This responds a DELETE request for the /del_user
page. app.delete('/del_user', function (req, res) {
  console.log("Got a DELETE request for
  /del_user"); res.send('Hello DELETE');
})
```

```
// This responds a GET request for the /list_user
page. app.get('/list_user', function (req, res) {
  console.log("Got a GET request for /list_user");
  res.send('Page Listing');
})
```

```
// This responds a GET request for abcd, abxcd, ab123cd, and so
on app.get('/ab*cd', function(req, res) {
  console.log("Got a GET request for
  /ab*cd"); res.send('Page Pattern Match');
})
```

```
var server = app.listen(8081, function
() { var host =
  server.address().address
  var port = server.address().port
```

```
  console.log("Example app listening at http://%s:%s", host, port)
```

})

Save the above code in a file named server.js and run it with the following command.

```
$ node server.js
```

You will see the following output –

Example app listening at

http://0.0.0.0:8081

Now you can try different requests at http://127.0.0.1:8081 to see the output generated by server.js. Following are a few screens shots showing different responses for

different URLs. Screen showing again http://127.0.0.1:8081/list_user



Screen showing again http://127.0.0.1:8081/abcd

Screen showing again http://127.0.0.1:8081/abcdefg





Serving Static Files

Express provides a built-in middleware **express.static** to serve static files, such as images, CSS, JavaScript, etc.

You simply need to pass the name of the directory where you keep your static assets, to the **express.static** middleware to start serving the files directly. For example, if you keep your images, CSS, and JavaScript files in a directory named public, you can do this –

```
app.use(express.static('public'));
```

We will keep a few images in **public/images** sub-directory as

follows – node_modules

server.js

public/

public/images

public/images/logo.png

Let's modify "Hello Word" app to add the functionality to handle

```
static files. var express = require('express');
```

```
var app = express();
```

```
app.use(express.static('publ
```

```
ic')); app.get('/', function
```

```
(req, res) {
```

```
  res.send('Hello World');
```

```
})
```

```
var server = app.listen(8081, function
```

```
() { var host =
```

```
  server.address().address
```



```
var port = server.address().port
```

```
  console.log("Example app listening at http://%s:%s", host, port)  
})
```

Save the above code in a file named server.js and run it with the following command.

```
$ node server.js
```

Now open <http://127.0.0.1:8081/images/logo.png> in any browser and see observe following result.



GET Method

Here is a simple example which passes two values using HTML FORM GET method. We are going to use **process_get** router inside server.js to handle this input.

```
<html>  
<body>  
  
  <form action = "http://127.0.0.1:8081/process_get" method =  
    "GET"> First Name: <input type = "text" name =  
      "first_name"> <br>  
      Last Name: <input type = "text" name = "last_name">  
      <input type = "submit" value = "Submit">  
  </form>  
  
</body>  
</html>
```

Let's save above code in index.htm and modify server.js to handle home page requests as well as the input sent by the HTML form.

```
var express =
```

```
require('express'); var app
= express();

app.use(express.static('public'));
app.get('/index.htm', function (req, res) {
  res.sendFile( __dirname + "/" + "index.htm" );
})
```

```
app.get('/process_get', function (req, res) {
  // Prepare output in JSON format
  response = {
    first_name:req.query.first_name,
    last_name:req.query.last_name
  };
  console.log(response);
  res.end(JSON.stringify(response));
})
```

```
var server = app.listen(8081, function
() { var host =
server.address().address
var port = server.address().port

  console.log("Example app listening at http://%s:%s", host, port)
})
```

Accessing the HTML document using <http://127.0.0.1:8081/index.htm> will generate the following form –

Top of Form

First Name:	
Last Name:	
<input type="submit" value="SUBMIT"/>	

Bottom of Form

Now you can enter the First and Last Name and then click submit button to see the result and it should return the following result –

```
{"first_name":"John","last_name":"Paul"} POST Method
```

Here is a simple example which passes two values using HTML FORM POST method. We are going to use **process_get** router inside server.js to handle this input.

```
<html>
  <body>

    <form action = "http://127.0.0.1:8081/process_post" method =
      "POST"> First Name: <input type = "text" name =
        "first_name"> <br>
      Last Name: <input type = "text" name = "last_name">
      <input type = "submit" value = "Submit">
    </form>

  </body>
</html>
```

Let's save the above code in index.htm and modify server.js to handle home page requests as well as the input sent by the HTML form.

```
var express =
require('express'); var app
= express();
var bodyParser = require('body-parser');

// Create application/x-www-form-urlencoded parser
var urlencodedParser = bodyParser.urlencoded({ extended: false })

app.use(express.static('public'));
app.get('/index.htm', function (req, res) {
  res.sendFile( __dirname + "/" + "index.htm" );
})

app.post('/process_post', urlencodedParser, function (req, res) {
  // Prepare output in JSON format
  response = {
    first_name:req.body.first_name,
    last_name:req.body.last_name
  };
  console.log(response);
  res.end(JSON.stringify(response));
})

var server = app.listen(8081, function
() { var host =
server.address().address
var port = server.address().port
```

```
console.log("Example app listening at http://%s:%s", host, port)
})
```

Accessing the HTML document using `http://127.0.0.1:8081/index.htm` will generate the following form –

Top of Form

<u>First</u> Name:	
<u>Last</u> Name:	
<u>SUBMIT</u>	

Bottom of Form

Now you can enter the First and Last Name and then click the submit button to see the following result –

```
{"first_name":"John","last_name":"Paul"}
```

File Upload

The following HTML code creates a file uploader form. This form has method attribute set to **POST** and enctype attribute is set to **multipart/form-data**

```
<html>
  <head>
    <title>File Uploading Form</title>
  </head>

  <body>
    <h3>File Upload:</h3>
    Select a file to upload:
    <br />

    <form action = "http://127.0.0.1:8081/file_upload" method =
      "POST" enctype = "multipart/form-data">
      <input type="file" name="file" size="50" />
      <br />
      <input type = "submit" value = "Upload File" />
    </form>

  </body>
</html>
```

Let's save above code in `index.htm` and modify `server.js` to handle home page requests as well as file upload.

```
var express =
require('express'); var app
= express();
var fs = require("fs");

var bodyParser = require('body-
parser'); var multer =
require('multer');

app.use(express.static('public'));
app.use(bodyParser.urlencoded({ extended: false
})); app.use(multer({ dest: '/tmp/'}));

app.get('/index.htm', function (req, res) {
  res.sendFile( __dirname + "/" +
  "index.htm" );
})

app.post('/file_upload', function (req, res)
{ console.log(req.files.file.name);
  console.log(req.files.file.path);
  console.log(req.files.file.type);
  var file = __dirname + "/" + req.files.file.name;

  fs.readFile( req.files.file.path, function (err, data) {
    fs.writeFile(file, data, function (err) {
      if( err ) {
        console.log( err
        );
      } else {
        response =
        {
          message:'File uploaded
          successfully',
          filename:req.files.file.name
        };
      }

      console.log( response );
      res.end( JSON.stringify( response ) );
    });
  });
})

var server = app.listen(8081, function
() { var host =
```

```
server.address().address  
var port = server.address().port
```

```
console.log("Example app listening at http://%s:%s", host, port)  
})
```

Accessing the HTML document using `http://127.0.0.1:8081/index.htm` will generate the following form –

File Upload:

Select a file to upload:

No file chosen

NOTE: This is just dummy form and would not work, but it must work at your server.

SERVER-SIDE RENDERING WITH TEMPLATING ENGINES

Templates - Server-Side Rendering

Regular Web Applications rely on templates to dynamically generate HTML pages on the server. These templates are text files that contain static content as well as a special syntax describing how the data should be inserted dynamically. During an HTTP request, the application loads and renders the template using the given contextual data and sends back the page to the client.

This technique is known as Server-Side Rendering (or SSR). Here is an example of what a template might look like:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% title %}</title>
</head>
<body>
  {% for user in users %}
    * {{ user.name }}
  {% else %}
    No users have been found.
  {% endfor %}
</body>

```

Rendering Templates

FoalTS provides a minimalist template engine to render templates. This engine replaces all the occurrences of `S myVariableName }}` with the given values.

Here is an example showing how to use it:

templates/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{ title }}</title>
</head>
<body>
  Hello {{ name }}!
</body>

```

src/app/app.controller.ts

```

import { Get, render } from '@foal/core';

export class AppController {
  @Get('/')
  index() {
    return render('./templates/index.html',
      { name: 'Alix',
        title: 'Home'
      });
  }
}

```

Output (GET /)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home</title>
</head>
<body>
  Hello
  Alix!
</body>
```

Using Another Template Engine

External template engines, such as EJS or pug, are also supported and can be configured for the current project using the configuration key `settings.templateEngine`.

Here is an example showing how to configure `config/default.json` (or `config/default.yml`) with twig, a JS implementation of the Twig PHP templating language.

```
npm install twig
```

```
module.exports = {
  settings: {
    templateEngine: "twig"
  }
}
```

Then the render function uses this engine under the hood to render the templates.

Note: Only Express compatible template engines are supported (which represents the large majority of those available on npm).

`templates/index.html` (Twig example)


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Users</title>
</head>
<body>
  <ul>
    {% for user in users %}
      <li>{{ user.name }}</li>
    {% endfor %}
  </ul>
</body>
```

src/app/app.controller.ts (Twig example)

```
import { Get } from '@foal/core';

export class AppController {
  @Get('/')
  index() {
    return render('./templates/index.html',
      { users: [
        { name: 'John' },
        { name: 'Mary' }
      ]
    });
  }
}
```

Output (GET /) (Twig example)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Users</title>
</head>
<body>
  <ul>
    <li>John</li>
    <li>Mary</li>
  </ul>
```

ExpressJS - Serving static files

Static files are files that clients download as they are from the server. Create a new directory, **public**. Express, by default does not allow you to serve static files. You need to enable it using the following built-in middleware.

```
app.use(express.static('public'));
```

Note – Express looks up the files relative to the static directory, so the name of the static directory is not part of the URL.

Note that the root route is now set to your public dir, so all static files you load will be considering public as root. To test that this is working fine, add any image file in your new **public** dir and change its name to "**testimage.jpg**". In your views, create a new view and include this file like –

html

head

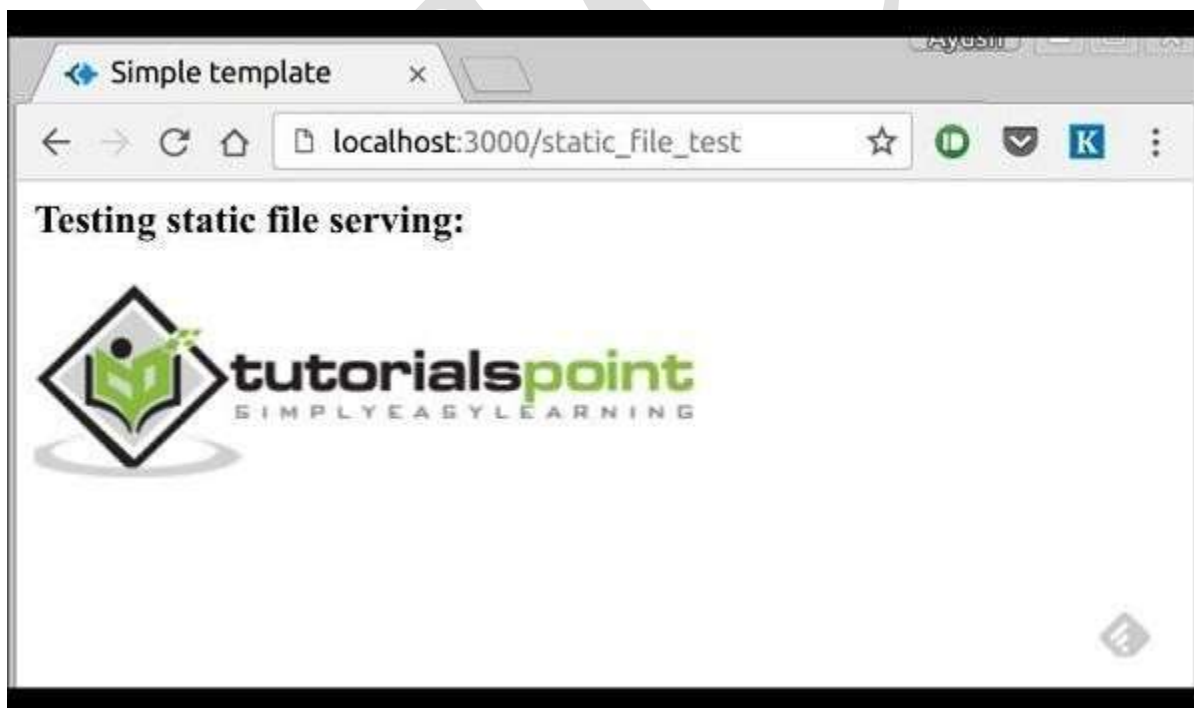
body

h3 Testing static file serving:

img(src = "/testimage.jpg", alt = "Testing

Image You should get the following output

–



Multiple Static Directories

We can also set multiple static assets directories using the following

```
program – var express = require('express');  
var app = express();
```

```
app.use(express.static('public'));
app.use(express.static('images'));
```

```
app.listen(3000);
```

Virtual Path Prefix

We can also provide a path prefix for serving static files. For example, if you want to provide a path prefix like **'/static'**, you need to include the following code in your **index.js** file –

```
var express =
require('express'); var app =
express();
```

```
app.use('/static',
```

```
express.static('public'));
```

```
app.listen(3000);
```

Now whenever you need to include a file, for example, a script file called main.js residing in your public directory, use the following script tag –

```
<script src = "/static/main.js" />
```

This technique can come in handy when providing multiple directories as static files. These prefixes can help distinguish between multiple directories.

Using Async Await in Node.js

Before Node version 7.6, the callbacks were the only official way provided by Node to run one function after another. As Node architecture is single-threaded and asynchronous, the community devised the callback functions, which would fire (or run) after the first function (to which the callbacks were assigned) run is completed.

Example of a Callback:

```
app.get('/', function(){
  function1(arg1,
  function(){
    ...
  })
});
```

The problem with this kind of code is that this kind of situations can cause a lot of trouble and the code can get messy when there are several functions. This situation is called what is commonly known as a callback hell. So, to find a way out, the idea of Promises and function chaining was introduced.

Example: Before async/await

```
function fun1(req, res){  
  return request.get('http://localhost:3000')  
    .catch((err) =>{  
      console.log('found  
      error');  
    }).then((res) =>{  
      console.log('get request returned.');    });
```

Explanation:

The above code demos a function implemented with function chaining instead of callbacks. It can be observed that the code is now more easy to understand and readable. The code basically says that GET localhost:3000, catch the error if there is any; if there is no error then implement the following statement:
console.log('get request returned.');

With Node v8, the async/await feature was officially rolled out by the Node to deal with Promises and function chaining. The functions need not to be chained one after another, simply await the function that returns the Promise. But the function async needs to be declared before awaiting a function returning a Promise. The code now looks like below.

Example: After async/await

```
async function fun1(req, res){  
  let response = await request.get('http://localhost:3000');  
  if (response.err) { console.log('error');} else { console.log('fetched response');  
}
```

Explanation:

The code above basically asks the javascript engine running the code to wait for the request.get() function to complete before moving on to the next line to execute it. The request.get() function returns a Promise for which user will await . Before async/await, if it needs to be made sure that the functions are running in the desired sequence, that is one after the another, chain them one after the another or register callbacks.

Code writing and understanding becomes easy with async/await as can be observed from both the examples.

FETCHING JSON FROM EXPRESS

Write the ex: program 4 th and explain it.

SNVCEET